

Constraint Satisfaction Problems

Cercheremo ancora soluzioni a problemi. Ora però gli stati assumono una struttura e contengono informazione che viene utilizzata nella ricerca di una soluzione. Si introduce la nozione di vincolo e si vede come alcuni metodi di ricerca già studiati possono essere applicati con successo. Si introducono anche nuovi metodi di ricerca, specifici.

Qualche esempio dell'uso di vincoli

- **Design di oggetti:**
la specifica della struttura di un oggetto deve rispettare dei vincoli funzionali e fisici
- **Allocazione di risorse:**
occorre tener conto di quantità, priorità, tempi di utilizzo, tempi per entrare nella disponibilità, politiche d'uso
- **Progettazione di circuiti:**
date un'occhiata a
<ftp://ftp.cs.brown.edu/pub/techreports/96/cs96-34.pdf>
- **Pianificazione percorsi robotici:**
tutti i movimenti fisici sono sottoposti a vincoli (pensate alle manovre per parcheggiare)



<https://www.flickr.com/photos/pio1976/3130250099>

Quali tipi di problemi?

- Un constraint satisfaction problem (CSP) è definito da:
 - Un insieme di variabili $X_1, \dots, X_n$
 - Un insieme di vincoli $C_1, \dots, C_m$
 - Opzionale: in alcuni casi è richiesta la massimizzazione di una funzione obiettivo

Stato del problema e assegnamento

- Gli **stati** di un CSP sono dati da tutti gli assegnamenti possibili per le variabili del CSP
- Un **assegnamento** $\{X_{i_1} = v_{i_1}, X_{i_2} = v_{i_2}, \dots\}$ è un'attribuzione di valori a un sottoinsieme delle variabili del CSP
- Un assegnamento è detto:
 - **Completo**: se assegna valori a tutte le variabili del CSP
 - **Consistente**: se non viola alcun vincolo del CSP
 - **Soluzione**: se è completo e consistente
- Quando esiste una soluzione per un vincolo si dice anche che esiste un mondo possibile che soddisfa il vincolo

Esempio: coloratura di mappe

Colorare la mappa di rosso, verde e blu evitando che territori confinanti siano dello stesso colore



Esempio: coloratura di mappe

7 variabili, una per territorio, i cui domini sono tutti uguali $\{R, G, B\}$

WA $\in \{R, G, B\}$

NT $\in \{R, G, B\}$

SA $\in \{R, G, B\}$

Q $\in \{R, G, B\}$

NSW $\in \{R, G, B\}$

V $\in \{R, G, B\}$

T $\in \{R, G, B\}$



Esempio: un possibile assegnamento

7 variabili, una per territorio, i cui domini sono tutti uguali $\{R, G, B\}$

WA = R

NT = R

SA = G

Q

NSW

V

T



Esempio: assegnamento consistente

7 variabili, una per territorio, i cui domini sono tutti uguali $\{R, G, B\}$

WA = R

NT = G

SA = B

Q

NSW

V

T



Esempio: una possibile soluzione

7 variabili, una per territorio, i cui domini sono tutti uguali {R, G, B}

WA = R

NT = G

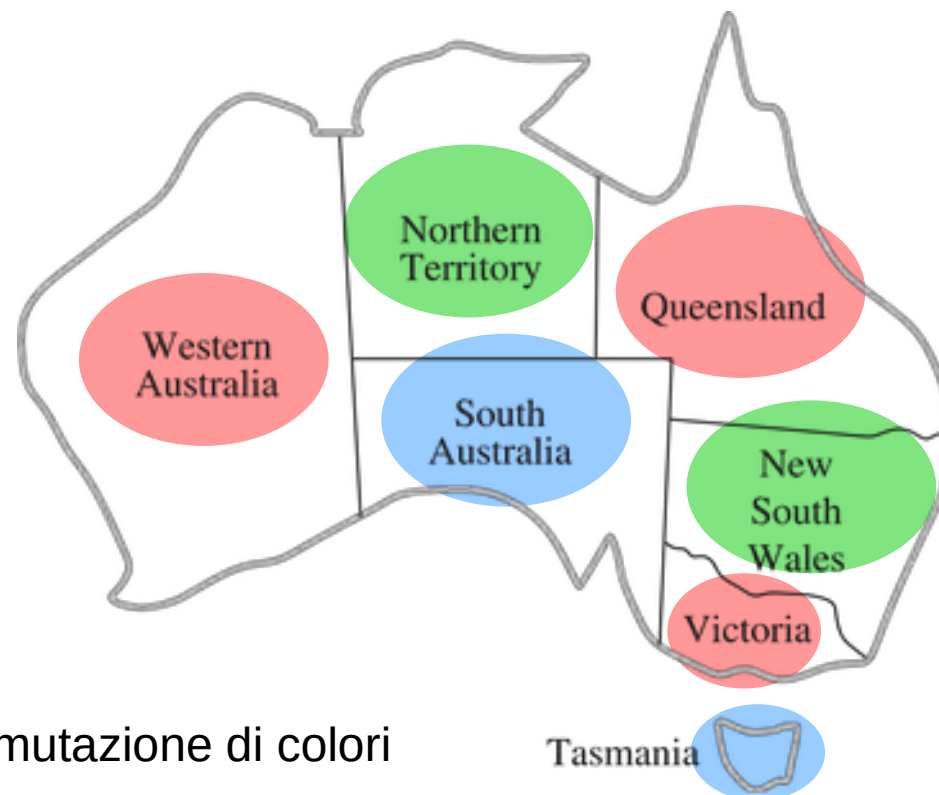
SA = B

Q = R

NSW = G

V = R

T = B



NB: qualunque permutazione di colori
è una soluzione

Esempio: vincoli

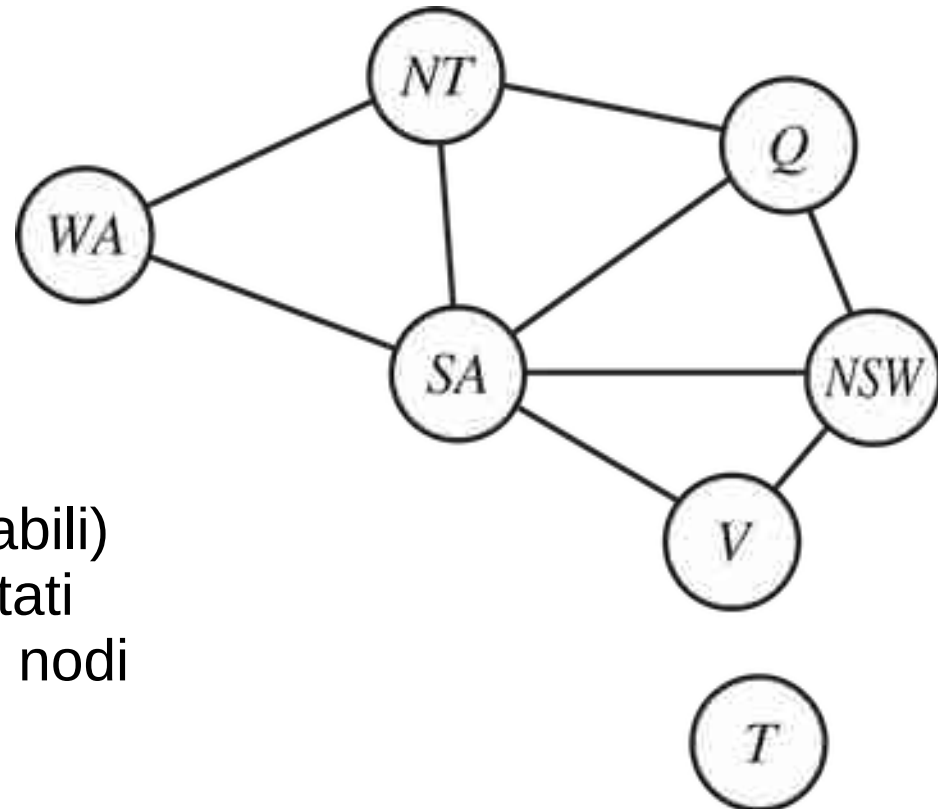
I vincoli riguardano coppie di territori confinanti. Supponendo che il linguaggio di rappresentazione contenga l'operatore $\neq$ (diverso) avremo quindi:

WA $\neq$ NT
WA $\neq$ SA
NT $\neq$ SA
NT $\neq$ Q
SA $\neq$ Q
SA $\neq$ NSW
SA $\neq$ V
Q $\neq$ NSW
NSW $\neq$ V



Tasmania è un'isola

Grafo di vincoli



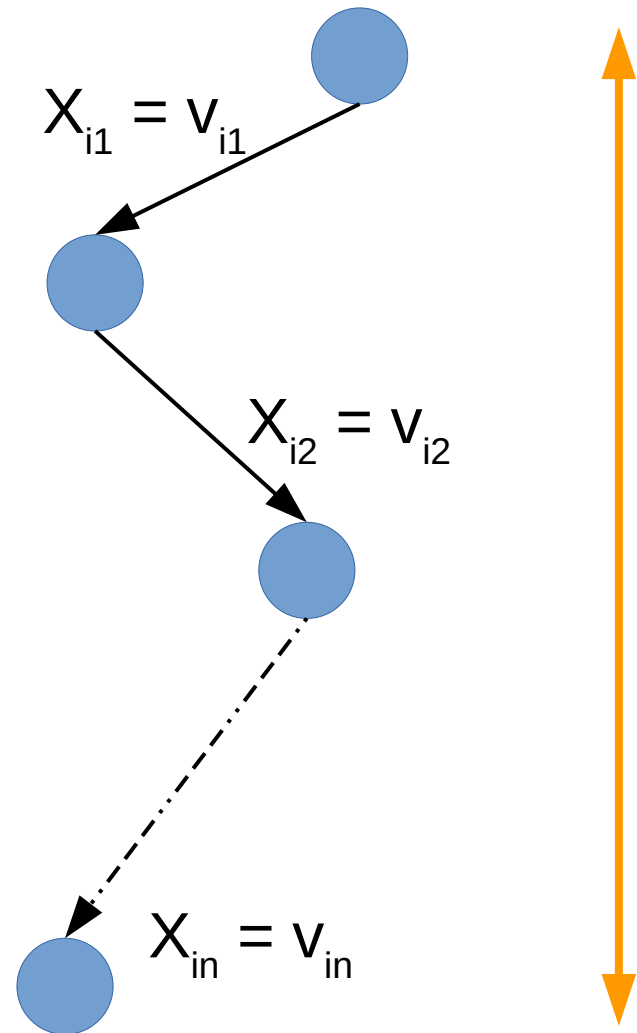
I vincoli binari (fra due variabili) possono essere rappresentati come archi di un grafo i cui nodi sono le variabili del CSP

CSP e problemi di ricerca

- È possibile formulare i CSP come problemi di ricerca in uno spazio degli stati:
 - **Stato iniziale** = { } assegnamento vuoto
 - **Successore** = assegnamento di un valore a una delle variabili che non ce l'hanno facendo attenzione che non sorgano conflitti
 - **Test obiettivo** = assegnamento completo
 - **Costo** = ogni passo ha costo costante (ad esempio 1)

Caratterizzazione

Profondità massima



Poiché ogni passo valorizza una variabile senza valore, i cammini saranno al più lunghi quanto il numero delle variabili del CSP. Sia n questo valore.

Stati dell'albero di ricerca

- Uno **stato** è un assegnamento di valori:
 $\{X1 = v1, X2 = v2, \dots, Xn = vn\}$
- L'**ordine** (il cammino) con cui i valori vengono assegnati alle variabili è irrilevante sul risultato, ad esempio:

$\{X1 = ?, \underline{X2 = v2}, X3 = ?\}$



$\{\underline{X1 = v1}, X2 = v2, X3 = ?\}$



$\{X1 = v1, X2 = v2, \underline{X3 = v3}\}$

$\{\underline{X1 = v1}, X2 = ?, X3 = ?\}$



$\{X1 = v1, X2 = ?, \underline{X3 = v3}\}$



$\{X1 = v1, \underline{X2 = v2}, X3 = v3\}$

Stesso risultato

Stati dell'albero di ricerca

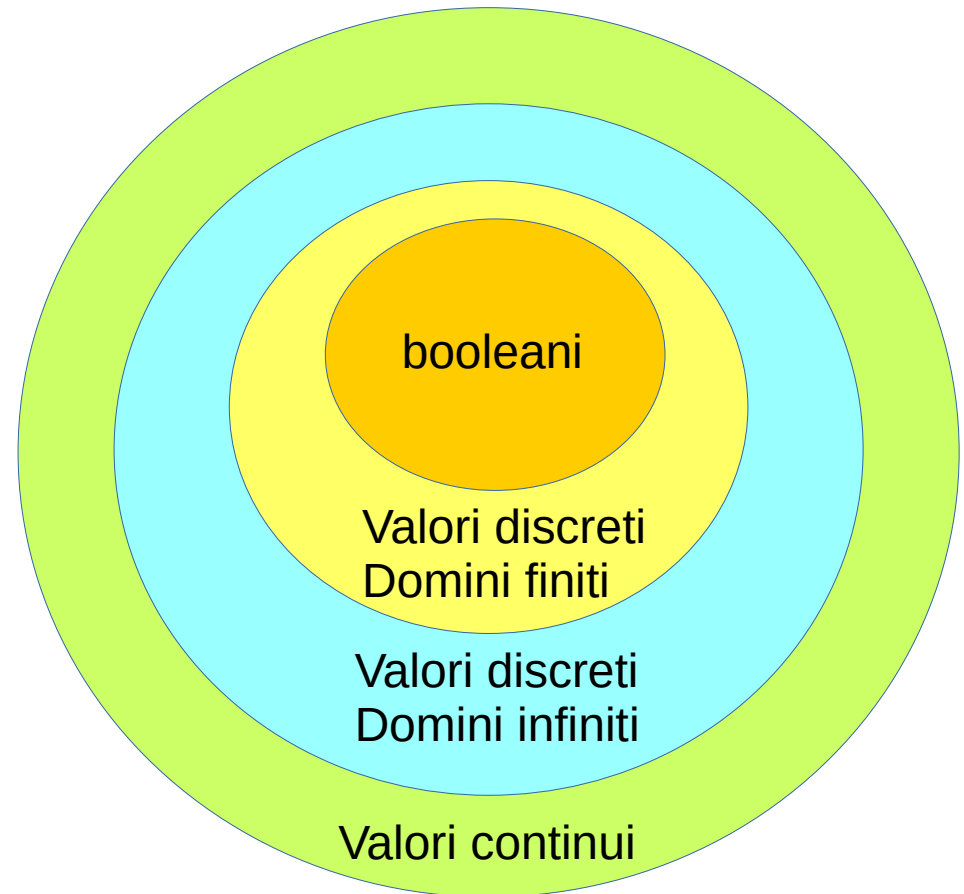
- Uno **stato** è un assegnamento di valori:
 $\{X_1 = v_1, X_2 = v_2, \dots, X_n = v_n\}$
- L'**ordine** (il **cammino**) con cui i valori vengono assegnati alle variabili è irrilevante sul risultato
- È possibile applicare metodi di ricerca già visti (esempio: blind)

Tipi di domini

- **Domini finiti:**
è possibile enumerare i vincoli mettendo in relazione i diversi valori (es. Australia)
- **Domini infiniti:**
non è possibile enumerare i vincoli,
si usano **linguaggi di specifica**, esempio per dire che Lavoro2 deve iniziare almeno 5 giorni dopo Lavoro1 si potrebbe scrivere: $\text{Lavoro1} + 5 < \text{Lavoro2}$
- **Domini continui**
la programmazione lineare permette di risolvere CSP in cui i **vincoli sono disuguaglianze lineari** che specificano una regione convessa

Variabili

- Possono essere:
 - A valori discreti
 - Booleani (NP-completa)
 - A domini finiti
 - A domini infiniti
 - A valori continui



Arità dei vincoli

- **Vincoli unari**

coinvolgono una variabile e un valore, esempio $T \neq G$ il colore della Tasmania deve essere diverso da verde

- **Vincoli binari**

coinvolgono due variabili e possono essere rappresentati come archi di un grafo, esempio $WA \neq NT$

- **Vincoli a tre o più variabili**

coinvolgono un numero qualsiasi di variabili, possono essere rappresentati da ipergraf (grafi con archi che connettono più di due nodi), esempio $diverse(WA, NT, SA)$. A volte possono essere scomposti in vincoli binari

Vincoli a 3 e più variabili, esempio

Vincoli a 3 e più variabili, esempio

- **Criptoaritmetica**: gioco in cui a ogni lettera corrisponde una cifra diversa, bisogna trovare la sostituzione corretta

$$\begin{array}{r} \text{S E N D} \\ + \text{M O R E} \\ \hline \text{M O N E Y} \end{array}$$

Vincoli a 3 e più variabili, esempio

- **Criptoaritmetica**: gioco in cui a ogni lettera corrisponde una cifra diversa, bisogna trovare la sostituzione corretta

$$\begin{array}{r} \text{S E N D} \\ + \text{M O R E} \\ \hline \text{M O N E Y} \end{array}$$

Dominio di S e M: $[1, 2, 3, 4, 5, 6, 7, 8, 9]$

Dominio di E, N, D, O, R, Y:

$[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]$

Vincoli: a lettere diverse valori diversi

Vincoli a 3 e più variabili, esempio

- **Criptoaritmetica**: gioco in cui a ogni lettera corrisponde una cifra diversa, bisogna trovare la sostituzione corretta

$$\begin{array}{r} \text{S E N D} \\ + \text{M O R E} \\ \hline \text{M O N E Y} \end{array}$$

questo vincolo coinvolge
tutte le variabili (non è
binario):

$$(1000*S + 100*E + 10*N + D + 1000*M + 100*O + 10*R + E) = (10000*M + 1000*O + 100*N + 10*E + Y)$$

Vincoli e criteri di preferenza

- I vincoli possono essere più o meno rigidi
- Distinguiamo fra **vincoli** veri e propri e **criteri di preferenza**
- Vincoli e criteri di preferenza si differenziano sul modo in cui la loro violazione impatta sulla soluzione:
 - Una soluzione **deve soddisfare tutti i vincoli**
 - Una soluzione **può violare uno o più criteri di preferenza**
 - Il soddisfacimento dei criteri di preferenza permette di ordinare le soluzioni identificando quelle preferibili e quelle meno preferibili

Esempio, criterio di preferenza

- Nella definizione dell'orario di lezione un docente può dire di preferire le lezioni del mattino
- Questo "vincolo" va composto con quelli degli altri docenti
- Risultato: l'orario prevede che il docente faccia lezione sempre al pomeriggio 😞

11-12	(Aula F)	Sist. AS, T2 (Laboratorio Dijkstra)	(Aula F)	(Aula A)	(Laboratorio Turing)
12-13	Sist. (Aula F)	Sist. AS, T2 (Laboratorio Von Neumann) Sist. AS, T2 (Laboratorio Dijkstra)	Sist. (Aula F)	Sist. AS (Aula A)	Prog. II Lab (Laboratorio Turing)
13-14					
14-15	Sist. Int (Aula F)	Logicatut (Aula C) Sist. (Aula F)	Sist. AS (Aula C)	Logicatut (Aula B) Sist. Int (Aula F)	Logicatut (Aula F)
15-16	Sist. Int (Aula F)	Logicatut (Aula C) Sist. (Aula F)	Sist. AS (Aula C)	Logicatut (Aula B) Sist. Int (Aula F)	Logicatut (Aula F)
16-17	Sist. AS, T2 (Laboratorio Von Neumann)				
17-18	Sist. AS, T2 (Laboratorio Von Neumann)				
18-19					

La soluzione non è ottimale, perché non soddisfa la preferenza, ma potrebbe essere la migliore fra quelle possibili

Rappresentare gli stati

- Stato = assegnamento
- Modi diversi di rappresentare lo stato influiscono sul tempo della ricerca
- Approccio di forza bruta: generate-and-test
Usa solo informazioni di stato
- Ricerca blind con backtracking
Usa anche i vincoli

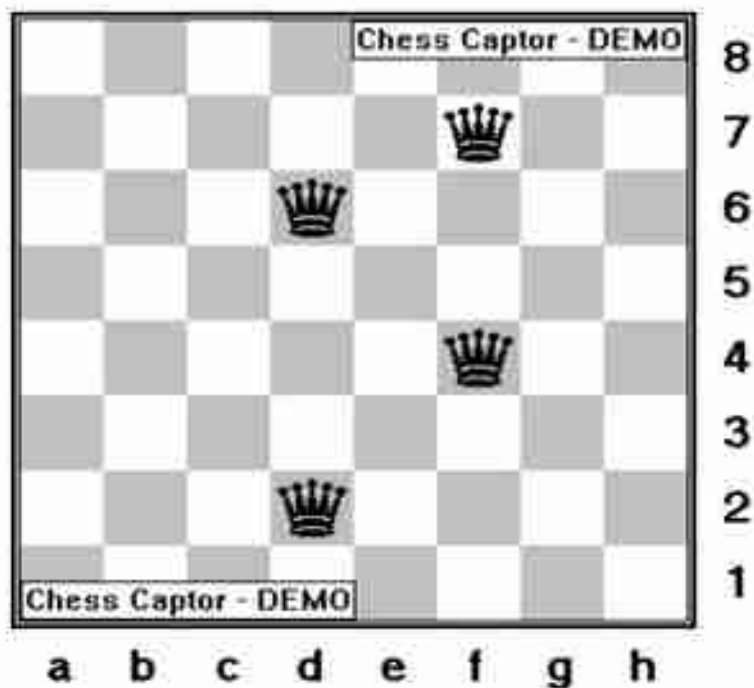
Usare solo informazioni di stato è pratico?

Generate-and-test

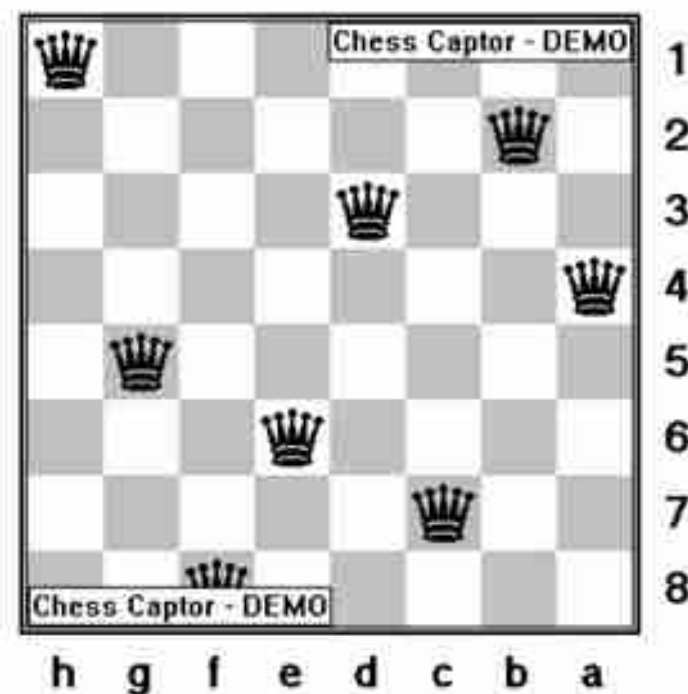
- Si può adottare una **rappresentazione a stato completo**, cioè utilizzare solo **assegnamenti completi**, che possono o meno essere consistenti
- **Generate-and-test**, metodo per la risoluzione di CSP molto semplice (ma costoso):
 - Finché non hai una soluzione e ci sono delle alternative:
 - 1) Genera un assegnamento completo
 - 2) Controlla se è consistente
 - 3) Se sì è una soluzione, esci dal ciclo
 - 4) Se no torna al passo 1
 - Se hai una soluzione restituiscila
 - Altrimenti fallimento
- Può richiedere l'esplorazione dell'intero spazio degli assegnamenti!

Esempio: problema delle 8 regine

- Posizionare 8 regine su una scacchiera 8x8 in modo tale che nessuna risulti sotto attacco di un'altra
- Il problema ha 92 soluzioni distinte (12 se si considerano solo quelle non ottenibili da altre per simmetria)



NO



SOLUZIONE



Generate-and-test: 8 regine

Q			
Q			
Q			
Q			

	Q		
Q			
Q			
Q			

		Q	
Q			
Q			
Q			

			Q
Q			
Q			
Q			

Q			
	Q		
Q			
Q			

Q			
		Q	
Q			
Q			

■ ■ ■

NB: le scacchiere sono 4x4 per rendere l'idea del procedimento senza occupare troppo spazio

Quanto tempo ci vorrà?

La risposta dipende da come abbiamo rappresentato il problema

8 regine come CSP

- **Primo modo (esempio):**
 - **Variabili:** $Q_1, Q_2, \dots, Q_8$
ognuna rappresenta la posizione di una regina
 - **Domini:** le posizioni sono numerate in modo crescente
 $1, 2, 3, \dots, 16, 17, 18, \dots, 64$
 - La soluzione riportata prima corrisponde all'assegnamento:
 $Q_1 = 1, Q_2 = 15, Q_3 = 21, Q_4 = 32, Q_5 = 34, Q_6 = 44, Q_7 = 54, Q_8 = 59$
 - **Problema:**
 - se ci sono **n variabili** ognuna delle quali può assumere **d valori**, si hanno **d^n possibili assegnamenti**
 - Nell'esempio ben oltre 28.100.000.000.000 possibili assegnamenti

8 regine come CSP, versione 2

- **Secondo modo:**
ogni regina deve per forza occupare una diversa colonna quindi **assegnamo ciascuna regina alla colonna di numero uguale** (esempio: Q1 sta nella colonna 1)
- **Variabili:** Q1, ..., Q8
- **Dominio:** 1, ... 8
il numero di riga che completa le coordinate della regina
- La soluzione di prima sarà quindi rappresentata come:
Q1 = 1, Q2 = 5, Q3 = 8, Q4 = 6, Q5 = 3, Q6 = 7, Q7 = 2, Q8 = 4
- **Va molto meglio:** abbiamo “solo” $8^8 = 16.777.216$ assegnamenti alternativi ma sono ancora tanti ...



Generate-and-test: 8 regine

- E se avessimo un numero maggiore di regine?
- In generale per N regine si hanno N^N configurazioni
- Se avessimo 16 regine le configurazioni sarebbero 18.446.744.073.709.551.616
- ~ 1200 anni di computazione

2) Usiamo più conoscenza: introduciamo i vincoli

Per passi, a partire da una semplice ricerca in profondità con backtracking, vedremo come si può rendere più efficiente la ricerca usando più e meglio l'informazione a disposizione

Generate-and-test: 8 regine

- Fino ad ora non abbiamo rappresentato i **vincoli**:
 - Date due qualsiasi regine Q_i e Q_j queste devono essere posizionate in modo che nessuna delle due sia attaccata dall'altra
 - Due regine si attaccano quando:
 - Occupano la stessa colonna:
impossibile nella nostra rappresentazione
 - Occupano la stessa riga:
aggiungiamo il vincolo $Q_i \neq Q_j$
 - Occupano la stessa diagonale:
aggiungiamo il vincolo $|i - j| \neq |Q_i - Q_j|$

Ricerca di una soluzione in profondità

- Esploriamo lo spazio degli stati (dei possibili assegnamenti) utilizzando una ricerca depth-first con backtracking
- Ricerca non informata + vincoli
per decidere quando potare un cammino
- La limitatezza dei cammini rende ragionevole la scelta della ricerca in profondità, tuttavia spesso branching factor elevato

Ricerca di una soluzione con backtracking

- Siano:
 - **n** = numero delle variabili
 - **d** = numero medio dei valori possibili per ciascuna variabile
 - Uno qualsiasi dei valori può essere assegnato a una qualsiasi delle variabili
- Il branching factor sarà **n*d** al primo livello, **(n-1)*d** al secondo (perché una variabile è stata fissata), eccetera
 - l'albero avrà **n! * dⁿ** foglie

Esempio Australia

7 variabili

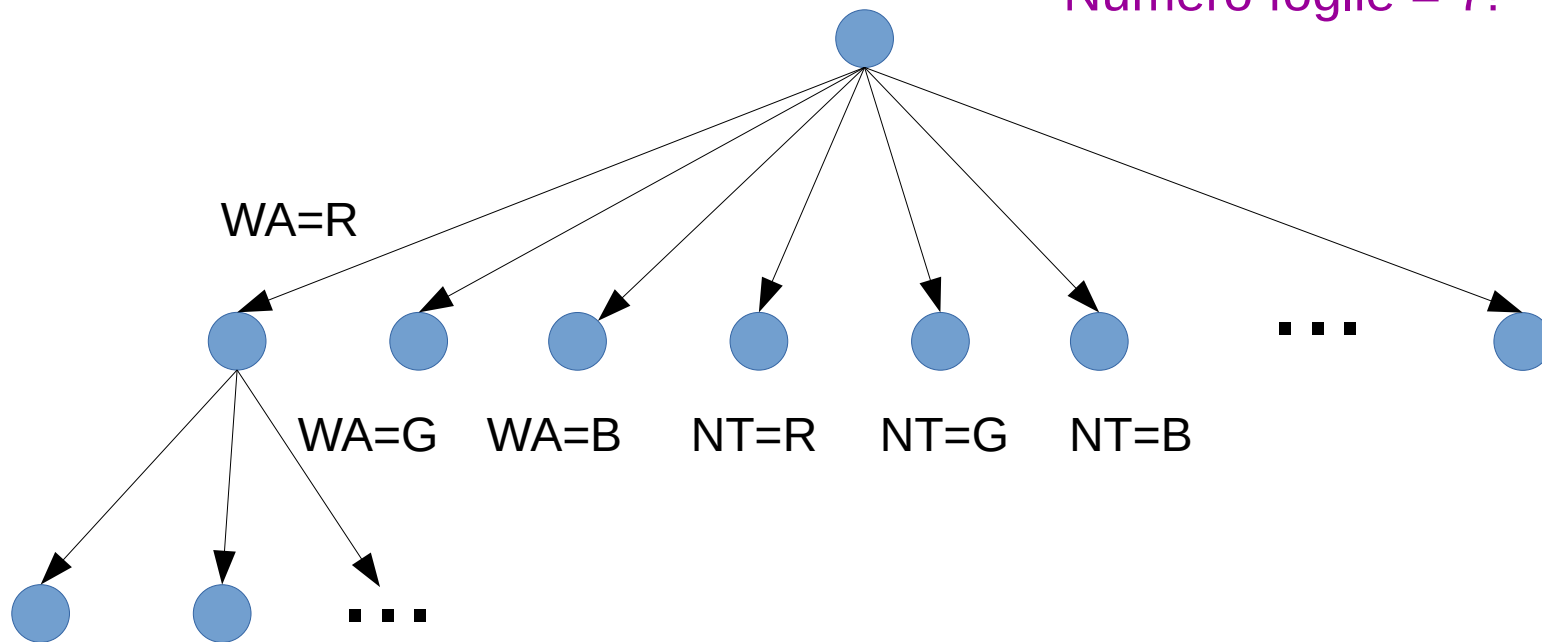
3 valori per variabile

Branching factor liv. 1 = 21

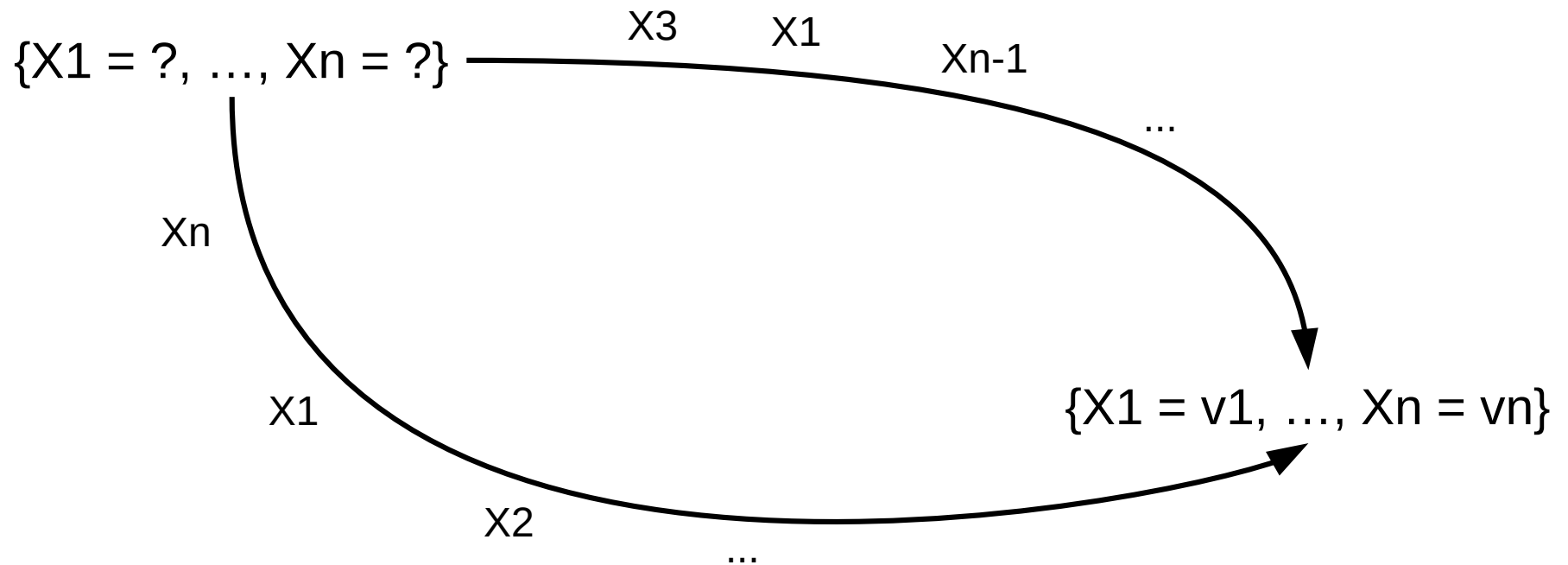
Branching factor liv. 2 = 18

□ □ □

Numero foglie = $7! \cdot 3^7 = 11.022.480$



I CSP sono commutativi

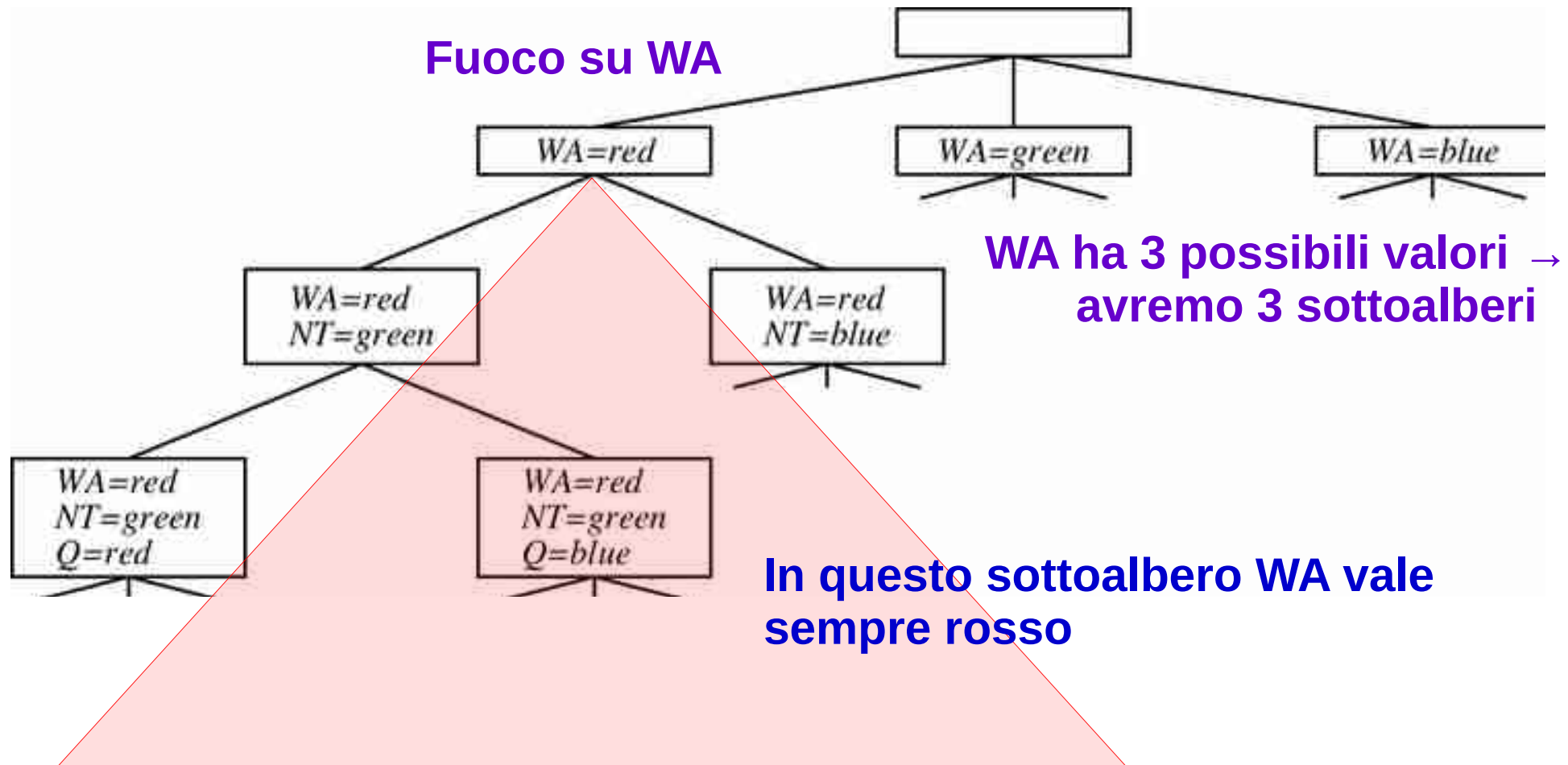


La stessa soluzione è ottenibile tramite più cammini, permutando l'ordine con cui le variabili vengono assegnate. La soluzione è sempre la stessa e l'ordine è ininfluenza sulla bontà della soluzione.

Gli algoritmi quindi **prima scelgono una variabile** e **poi un valore** per questa variabile

Esempio, Australia

Fuoco su WA



Con questa restrizione **il numero di foglie si riduce a d^n** , nel nostro esempio a **2.187** (un bel risparmio rispetto a 11.022.480)

Ricerca con backtracking: algoritmo

```
function BACKTRACKING-SEARCH(csp) returns a solution, or failure
  return RECURSIVE-BACKTRACKING({ }, csp)

function RECURSIVE-BACKTRACKING(assignment, csp) returns a solution, or failure
  if assignment is complete then return assignment
  var ← SELECT-UNASSIGNED-VARIABLE(VARIABLES[csp], assignment, csp)
  for each value in ORDER-DOMAIN-VALUES(var, assignment, csp) do
    if value is consistent with assignment according to CONSTRAINTS[csp] then
      add { var = value } to assignment
      result ← RECURSIVE-BACKTRACKING(assignment, csp)
      if result ≠ failure then return result
      remove { var = value } from assignment
  return failure
```

Figure 5.3 A simple backtracking algorithm for constraint satisfaction problems. The algorithm is modeled on the recursive depth-first search of Chapter 3. The functions SELECT-UNASSIGNED-VARIABLE and ORDER-DOMAIN-VALUES can be used to implement the general-purpose heuristics discussed in the text.

Questo algoritmo è riportato nella II edizione del libro ma non nella III

Ricerca con backtracking: algoritmo

```
function BACKTRACKING-SEARCH(csp) returns a solution, or failure
  return RECURSIVE-BACKTRACKING({ }, csp)

function RECURSIVE-BACKTRACKING({ }, assignment, csp) returns failure
  if assignment is complete then return assignment
  var ← SELECT-UNASSIGNED-VARIABLE(VARIABLES[csp], assignment, csp)
  for each value in ORDER-DOMAIN-VALUES(var, assignment, csp) do
    if value is consistent with assignment according to CONSTRAINTS[csp] then
      add { var = value } to assignment
      result ← RECURSIVE-BACKTRACKING({ }, assignment, csp)
      if result ≠ failure then return result
      remove { var = value } from assignment
  return failure
```

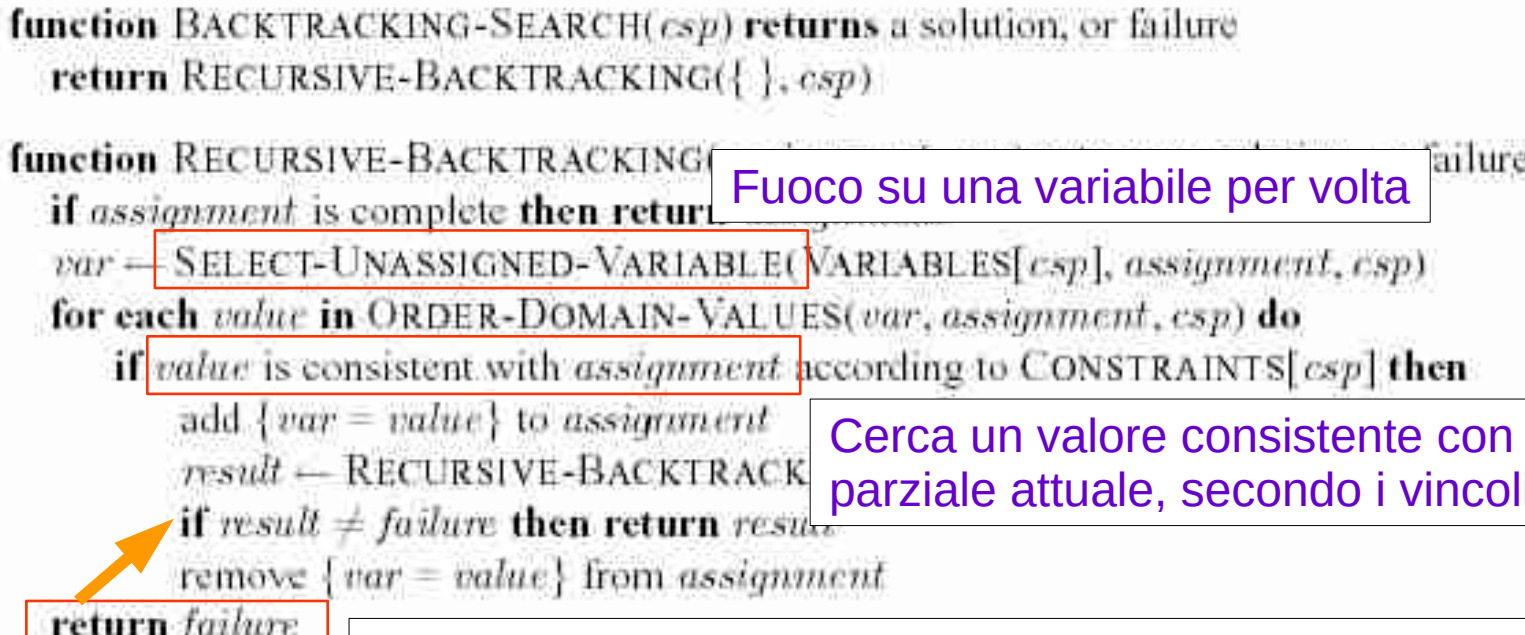


Figure 5.3 A

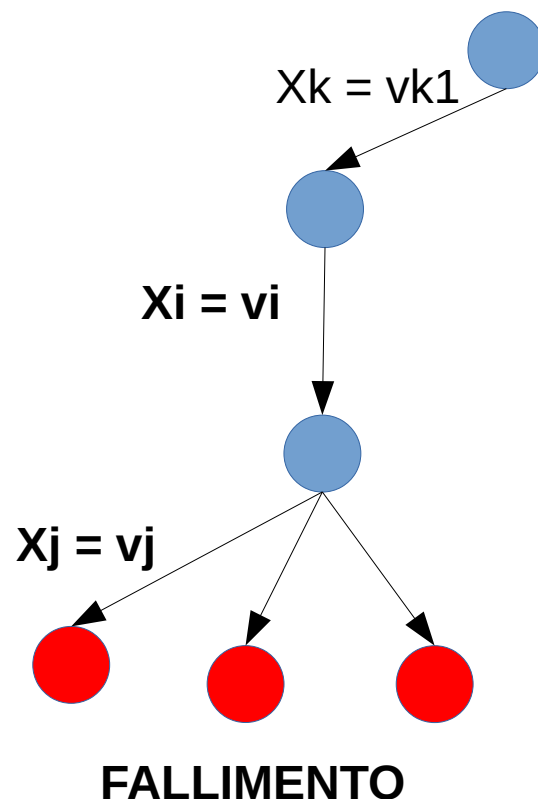
algorithm is modeled on the recursive depth-first search of Chapter 3. The functions SELECT-UNASSIGNED-VARIABLE and ORDER-DOMAIN-VALUES can be used to implement the general-purpose heuristics discussed in the text.

Ricerca con backtracking: osservazioni

- È una ricerca in profondità, non informata
- Non è particolarmente efficiente, in particolare è soggetta a **thrashing** (lo stesso assegnamento errato può essere ripetuto in più punti dell'albero), esempio:

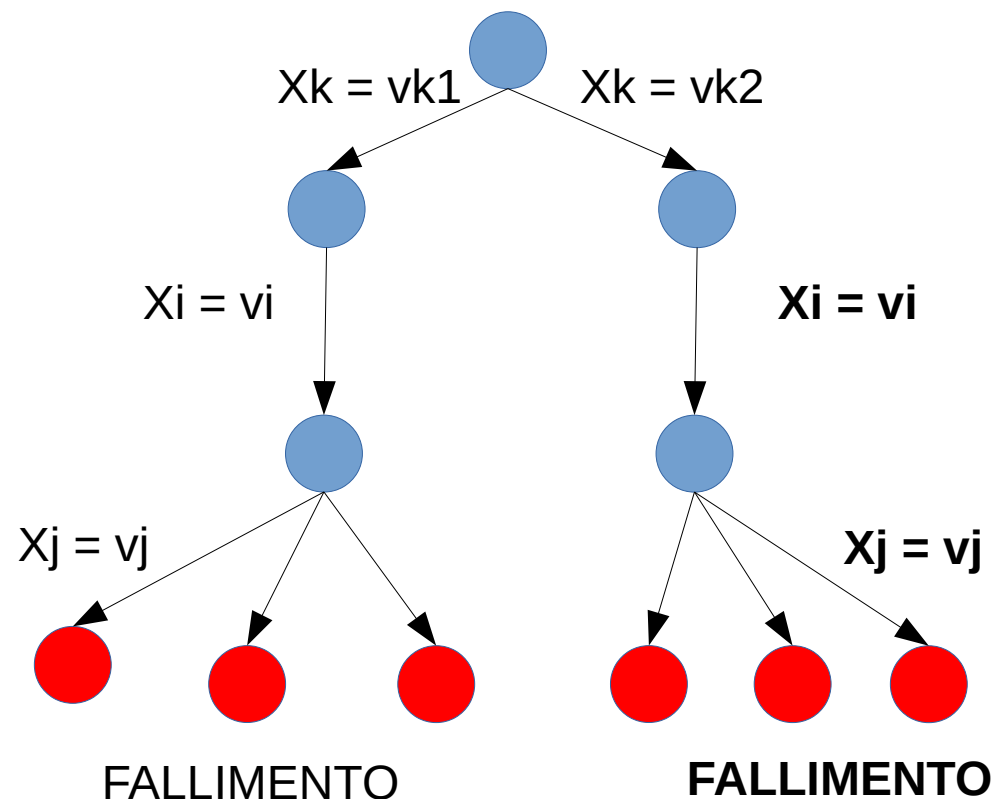
Ricerca con backtracking: osservazioni

- Supponiamo che il valore v_i di X_i sia incompatibile e quindi disabiliti tutti i valori di X_j . Questo sottoalbero è destinato a fallire



Ricerca con backtracking: osservazioni

- Se si fosse costretti a esplorare altri sottoalberi in cui nuovamente si considerano prima X_i e poi X_j si avrebbero altri fallimenti:



Ricerca con backtracking: osservazioni

- È una ricerca in profondità, non informata
- Non è particolarmente efficiente
- **Direzioni di miglioramento richiedono conoscenza sui vincoli**
 - 1) **L'assegnamento della variabile corrente** ha un impatto sulle variabili non ancora assegnate?
 - 2) **L'ordinamento dei valori** da provare influisce sul risultato?
 - 3) Quando un cammino fallisce, è possibile fare tesoro dell'esperienza per **evitare fallimenti simili** lungo altri cammini?

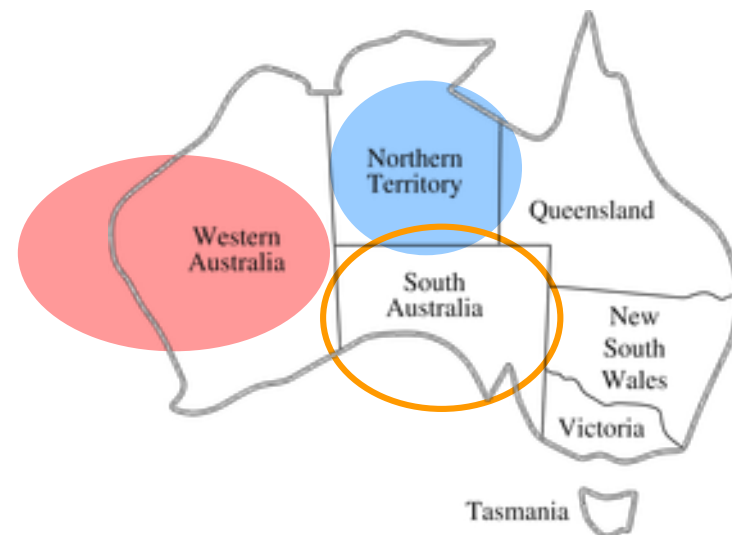
Euristiche di scelta della variabile

- L'algoritmo con backtracking sceglie, a ogni iterazione una variabile non assegnata:
 - *Finora abbiamo attuato scelte casuali*
 - *Se invece di una scelta casuale si adotta un'**euristica di scelta** si possono migliorare le prestazioni*
- **Esempio:** supponiamo si sia scelto il rosso per WA e il blu per NT, rimangono 5 variabili da assegnare
- Quale è più sensato scegliere?



Euristiche di scelta della variabile

- *La più vincolata*, cioè SA per la quale esiste uno solo valore possibile
- SA ha il vincolo di *non essere blu o rossa*, Q ha il vincolo di *non essere blu*, gli altri territori al momento potrebbero essere di qualsiasi colore
- **Euristica Minimum Remaining Values (o fail-first)**: sceglie una delle variabili con il minor numero di valori alternativi consistenti con l'assegnamento corrente



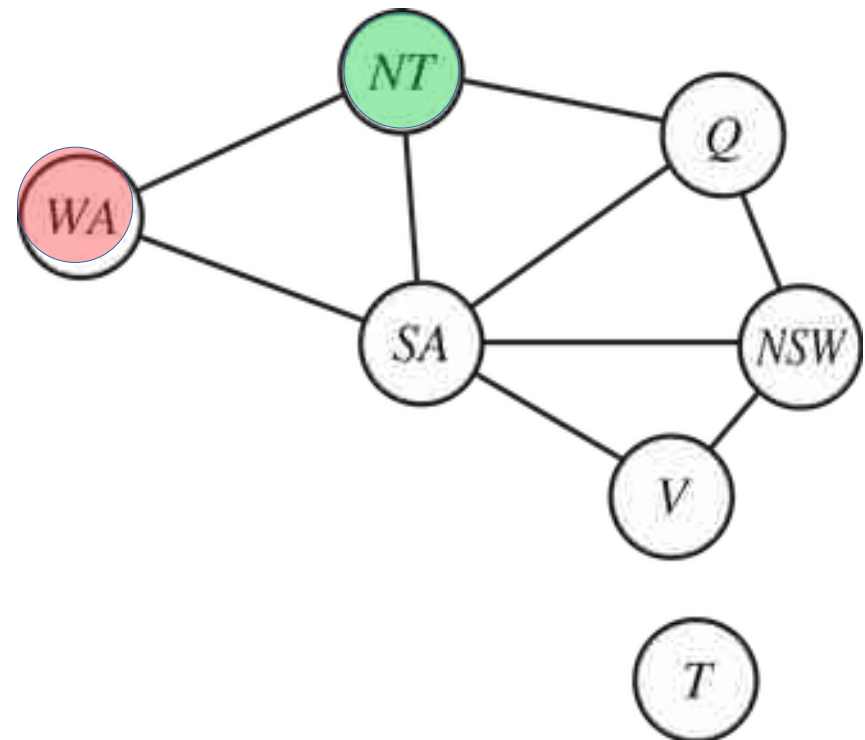
Euristiche di scelta della variabile

- *MRV non è sempre di aiuto (esempio, nella scelta della prima variabile da assegnare o in generale in caso di pareggio)*
- *In questo caso si usa l'**euristica di grado** che sceglie la variabile coinvolta in più vincoli*
- nell'esempio, se l'assegnamento iniziale fosse solo $NT=B$:
 - WA, SA e Q sarebbero a parimerito per MRV con due valori consistenti
 - l'euristica di grado sceglierebbe SA che è coinvolta in 5 vincoli

Euristiche di scelta del valore

- *Scelta una variabile che può assumere diversi valori, quale valore considerare prima?*
- **Euristica del valore meno vincolante:** prediligere il valore che lascia più libertà alle variabili adiacenti sul grafo dei vincoli

Supponiamo di avere assegnato **WA=R** e **NT=G** e di dover assegnare **Q**



Euristica del valore meno vincolante

Valori possibili per Q: **B** e **R**

Valori possibili per i vicini di Q:

SA: **B**

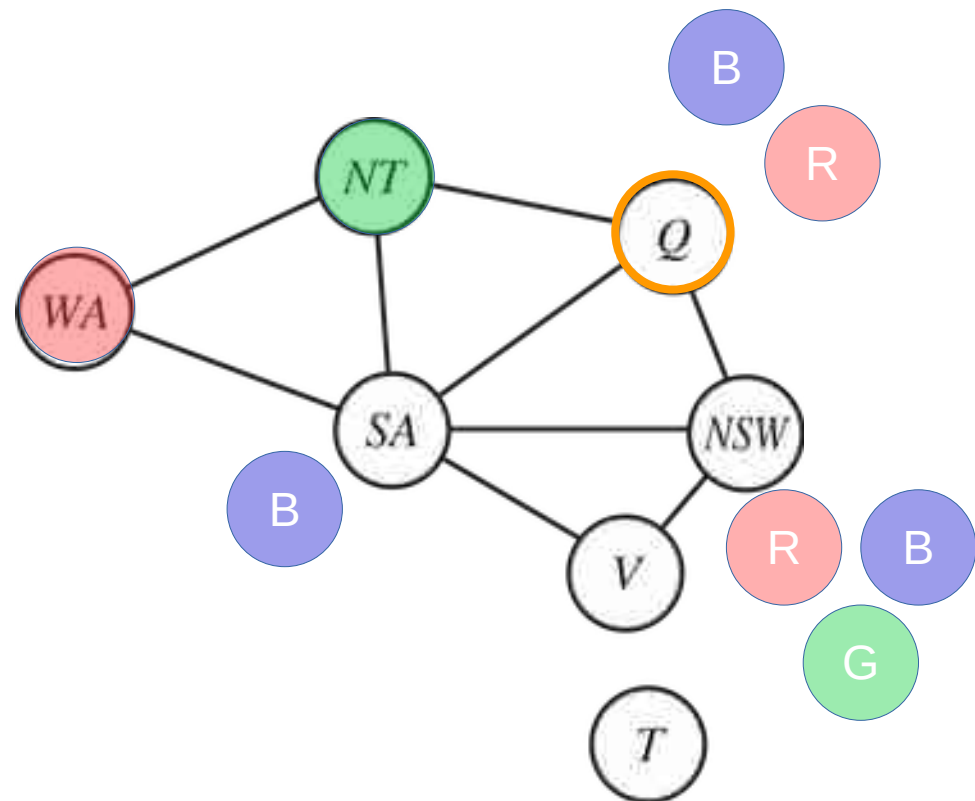
NSW: **R**, **G**, **B**

Se Q assume valore B: SA non può più assumere valori e i valori di NSW si riducono a 2

Se Q assume valore R: i valori possibili per SA non sono ridotti mentre NSW potrà essere G o B

È meno vincolante $Q=R$

Una scelta restrittiva anticipata rischia di tagliare via soluzioni



Rendiamo informata la ricerca

- La ricerca nello spazio dei possibili assegnamenti può essere ulteriormente migliorata usando meglio la conoscenza data dai vincoli
- Si alternano fasi di esplorazione a fasi di inferenza
- L'inferenza è espressa in termini di propagazione dei vincoli

Backtracking con inferenza

```
function RICERCA-BACKTRACKING(csp) returns una soluzione, o il fallimento
  return BACKTRACKING({ }, csp)

function BACKTRACKING(assegnamento, csp) returns una soluzione, o il fallimento
  if assegnamento è completo then return assegnamento
  var ← SCEGLI-VARIABILE-NON-ASSEGNATA(csp)
  for each valore in ORDINA-VALORI-DOMINIO(var, assegnamento, csp) do
    if valore è consistente con assegnamento then
      aggiungi {var = valore} ad assegnamento
      inferenze ← INFERENZA(csp, var, valore)
      if inferenze ≠ fallimento then
        aggiungi inferenze ad assegnamento
        risultato ← BACKTRACKING(assegnamento, csp)
        if risultato ≠ fallimento then
          return risultato
      rimuovi {var = valore} e inferenze da assegnamento
  return fallimento
```

Figura 6.5 Un semplice algoritmo di backtracking per problemi di soddisfacimento di vincoli. L'algoritmo

Backtracking con inferenza

```
function RICERCA-BACKTRACKING(csp) returns una soluzione, o il fallimento
  return BACKTRACKING({ }, csp)

function BACKTRACKING(assegnamento, csp) returns una soluzione, o il fallimento
  if assegnamento è completo then return assegnamento
  var ← SCEGLI-VARIABILE-NON-ASSEGNATA(csp)
  for each valore in ORDINA-VALORI-DOMINIO(var, assegnamento, csp) do
    if valore è consistente con assegnamento then
      aggiungi {var = valore} ad assegnamento
      inferenze ← INFERENZA(csp, var, valore)
      if inferenze ≠ fallimento then
        aggiungi inferenze ad assegnamento
        risultato ← BACKTRACKING(assegnamento, csp)
        if risultato ≠ fallimento then
          return risultato
      rimuovi {var = valore} e inferenze da assegnamento
  return fallimento
```

EURISTICHE

Figura 6.5 Un semplice algoritmo di backtracking per problemi di soddisfacimento di vincoli. L'algoritmo

Consistenza locale

- **Scopo:** *ridurre lo spazio di ricerca in modo consistente usando i vincoli per tagliare assegnamenti non consistenti con l'informazione disponibile*
- **Metodo di consistenza locale:**
focalizzarsi su una **proprietà di consistenza relativa ad una parte del CSP** (un sottoproblema) e poi propagare questa informazione in tutto il CSP cercando di costruire una soluzione
- Fra le proprietà considerate:
 - 1) **Forward checking** (è un approccio più che una proprietà)
 - 2) **Node consistency:** riguarda singole variabili, vale quando i vincoli unari sono soddisfatti da tutti i valori dei domini delle rispettive variabili
 - 3) **Arc consistency:** proprietà direzionale relativa a un vincolo binario (vediamo più avanti), tutti i valori consistenti di una variabile x possono essere estesi a y tramite i vincoli
 - 4) **Path consistency:** proprietà che lega una coppia di variabili a una terza tramite i vincoli (vedremo più avanti)

Forward checking

- **Osservazione:** se $WA=R$ a causa dei vincoli le confinanti NT e SA non potranno assumere questo valore
- **Forward checking:**
si percorrono gli archi che collegano il nodo, corrispondente alla variabile assegnata, con i suoi vicini diretti e si riduce il range dei possibili valori di tali vicini in maniera conforme al vincolo
- **Esempio:**
quando WA diventa R si toglie R dai valori possibili di NT e SA
- Il forward checking si accompagna naturalmente con l'euristica MRV

Esempio

- La tabella riporta riduzioni progressive dei domini delle variabili. Vengono propagate dai vincoli attraverso al grafo dei vincoli a seguito degli assegnamenti provati. Quando una variabile (SA) non ha più valori legali si ha il fallimento e di conseguenza il backtracking

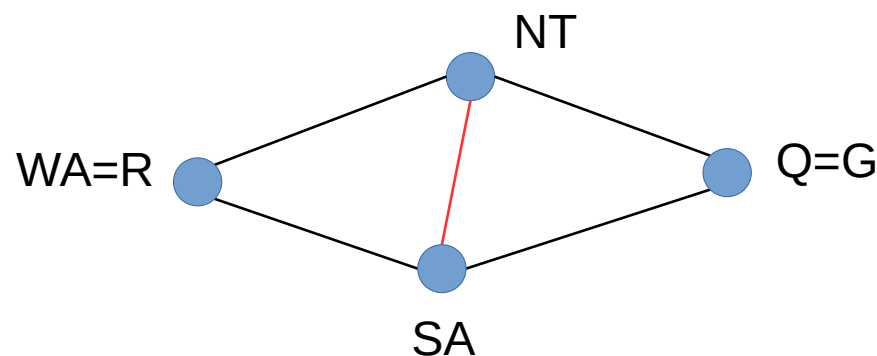
	WA	NT	Q	NSW	V	SA	T
INIZIO	R G B	R G B	R G B	R G B	R G B	R G B	R G B
WA=R →	R	G B	R G B	R G B	R G B	G B	R G B
Q=G →	R	B	G	R B	R G B	B	R G B
V=B →	R	B	G	R	B		R G B

Confinano tutti con SA

Limiti del forward checking

- Il forward checking non cattura tutte le inconsistenze

WA	NT	Q	NSW	V	SA	T
R G B	R G B	R G B	R G B	R G B	R G B	R G B
R	G B	R G B	R G B	R G B	G B	R G B
R	B	G	R B	R G B	B	R G B
R	B	G	R	B		R G B



Questo assegnamento forza un'inconsistenza infatti NT e SA, confinanti sono richiesti avere lo stesso valore!!

Forward checking non se ne accorge

Oltre il forward checking

- Node consistency
- Arc consistency
- Path consistency

Node consistency

- Un grafo di vincoli è **node consistent** quando lo sono tutte le sue variabili
- Una variabile è **node consistent** quando rispetta tutti i vincoli unari che la riguardano
- **Esempio:** “età compresa fra 18 e 35 anni” può essere espresso con due vincoli unari: $\text{età} > 18$ e $\text{età} < 35$
- L'algoritmo di node consistency controlla che un grafo di vincoli sia node consistent valutando tutte le variabili e i loro domini correnti rispetto ai vincoli unari

Arc consistency

- Un grafo di vincoli è **arc consistent** quando rispetta tutti i vincoli binari
- È una proprietà importante perché: dato un vincolo a tre o più variabili è sempre possibile trasformarlo in un insieme di vincoli binari
- **NB:** il forward checking realizza un controllo di arc consistency per i vincoli che legano una variabile, alla quale è stato assegnato un valore, e i suoi vicini diretti sul grafo dei vincoli ma non propaga il ragionamento sui vicini dei vicini, ecc.

Arc consistency

- Dato un grafo di vincoli si intende per **arco** un *lato orientato*
- **Esempio:** $WA \rightarrow NSW$ sarà diverso da $NSW \rightarrow WA$
- Un arco è **consistente** quando:
per ogni valore possibile della variabile corrispondente al vertice sorgente, esiste un qualche valore del vertice destinazione che è ad esso consistente
- **Esempio:**
 - se il dominio di **WA** è $\{R, G\}$ e il dominio di **NSW** è $\{R\}$
 - **WA** $\rightarrow$ **NSW** non è consistente
se WA assume valore R, NSW non ha valori assegnabili
 - **NSW** $\rightarrow$ **WA** è consistente
- Si può cercare di rendere consistenti gli archi che non lo sono cancellando valori dai loro domini

- Algoritmo di arc consistency sviluppato nel 1977 da Alan Macworth[§] (AC-3 = Arc Consistency algorithm #3)
- Viene insegnato perché è più efficiente dei precedenti e più semplice dei successori
- Si può usare come **preprocessing** oppure a valle degli assegnamenti per **propagare le scelte** fatte tramite i vincoli. Quest'ultimo uso realizza l'algoritmo MAC (Maintaining Arc Consistency)

[§] se può essere utile sono accessibili delle slide introduttive ai CSP dello stesso Mackworth:
<http://www.cs.ubc.ca/~mack/CS322/lectures/3-CSP2.pdf>

function AC-3(*esp*) **returns** the CSP, possibly with reduced domains

inputs: *esp*, a binary CSP with variables $\{X_1, X_2, \dots, X_n\}$

local variables: *queue*, a queue of arcs, initially all the arcs in *esp*

while *queue* is not empty **do**

$(X_i, X_j) \leftarrow \text{REMOVE-FIRST}(\text{queue})$

if REMOVE-INCONSISTENT-VALUES(X_i, X_j) **then**

for each X_k **in** NEIGHBORS[X_i] **do**

 add (X_k, X_i) to *queue*

function REMOVE-INCONSISTENT-VALUES(X_i, X_j) **returns** true iff we remove a value

removed $\leftarrow$ false

for each x **in** DOMAIN[X_i] **do**

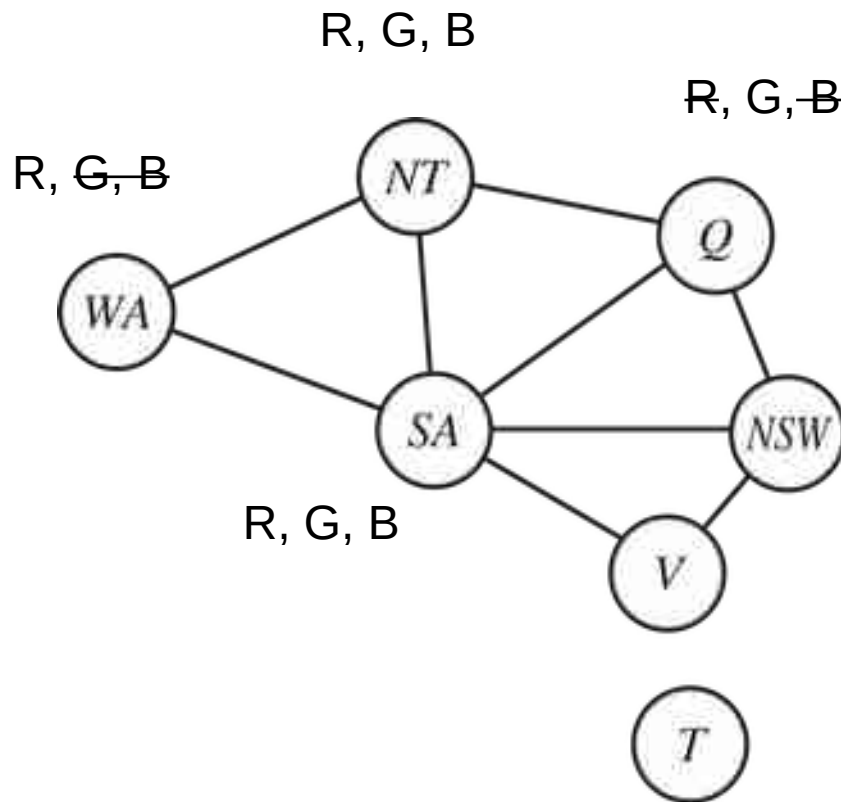
if no value y in DOMAIN[X_j] allows (x, y) to satisfy the constraint between X_i and X_j

then delete x from DOMAIN[X_i]; *removed* $\leftarrow$ true

return *removed*

Figure 5.7 The arc consistency algorithm AC-3. After applying AC-3, either every arc is arc-consistent, or some variable has an empty domain, indicating that the CSP cannot be made arc-consistent (and thus the CSP cannot be solved). The name “AC-3” was used by the algorithm’s inventor (Mackworth, 1977) because it’s the third version developed in the paper.

Esempio



VARIABILI: WA, NT, SA, Q, ...

DOMINI PER LE VRB NON ASSEGNATE:
{R, G, B}

QUEUE:

~~WA~~ → NT NT → WA
NT → Q Q → NT
WA → SA SA → WA
NT → WA WA → NT
SA → Q Q → SA
... ecc ...

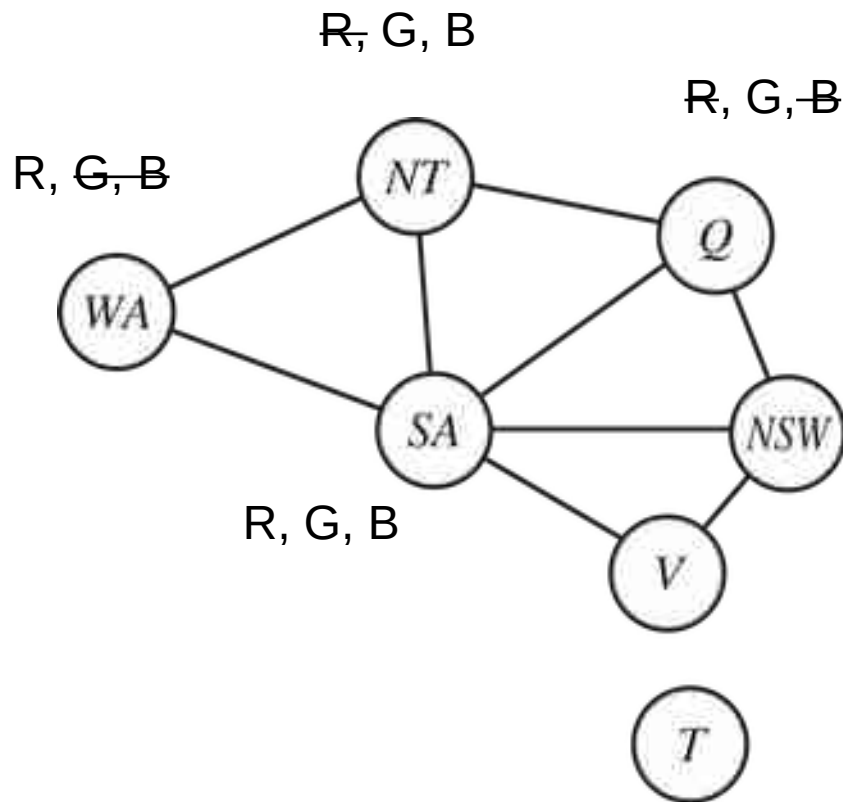
Supponiamo di avere assegnato **WA=R, Q=G**
Gli archi sono consistenti?

Seleziono WA → NT

È arc-consistent (il valore R di WA è consistente con i valori G e B di NT)

Togliamo l'arco da queue

Esempio



QUEUE:

~~WA~~ → NT NT → ~~WA~~
 NT → Q Q → NT
 WA → SA SA → WA
 NT → WA WA → NT
 SA → Q Q → SA,
WA → **NT**, ... ecc ...

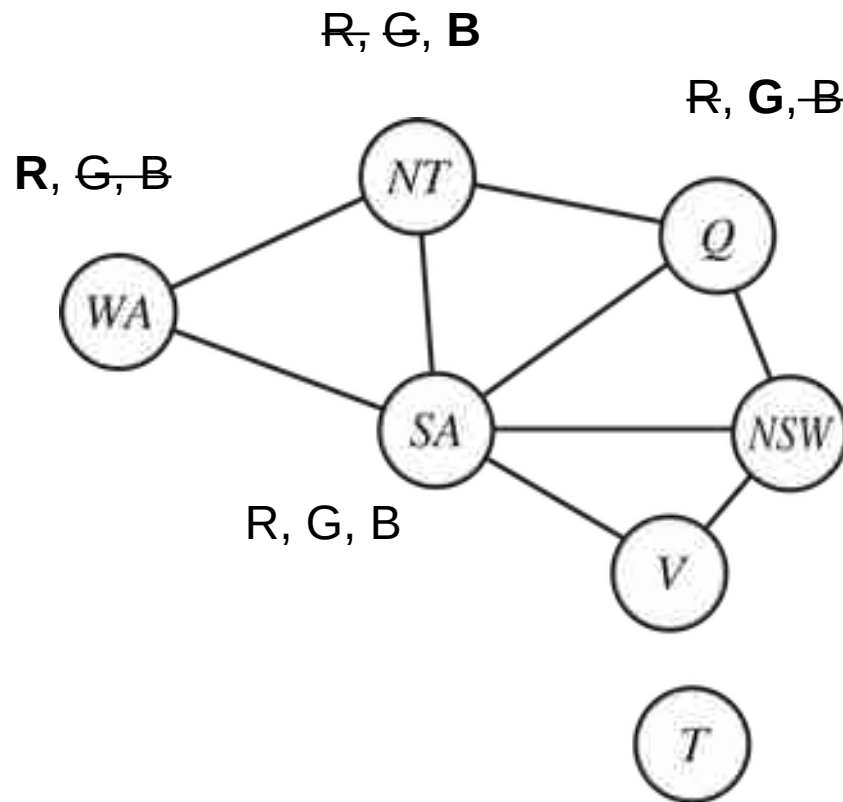
Seleziono NT → WA

Non è arc-consistent:
 il valore R di NT è inconsistente con
 il valore R di WA

Togliamo R dai valori possibili di NT

Togliamo NT → WA da queue ma aggiungiamo
 gli archi di tipo NODO → NT, quindi:
WA → **NT** (rientra, era stato rimosso),
 SA → NT, Q → NT (ci sono già)

Esempio



QUEUE:

~~WA~~ → NT NT → ~~WA~~
NT → Q Q → NT
WA → SA SA → WA
NT → WA WA → NT
SA → Q Q → SA,
WA → **NT**, ... ecc ...

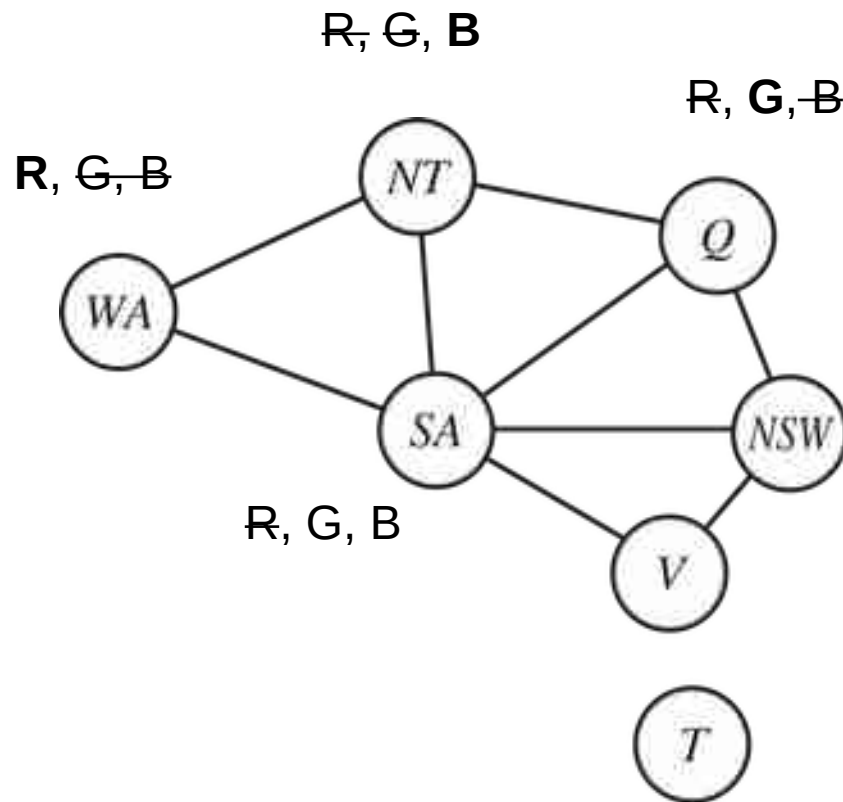
Seleziono NT → Q

Non è arc-consistent:
il valore G di NT è inconsistente con
il valore G di Q

Togliamo G dai valori possibili di NT

Togliamo NT → Q da queue ma aggiungiamo
gli archi di tipo NODO → NT. Questi sono
però già presenti, li abbiamo aggiunti all'itera-
zione precedente e non ancora esaminati

Esempio



QUEUE:

~~WA $\rightarrow$ NT~~ ~~NT $\rightarrow$ WA~~
~~NT $\rightarrow$ Q~~ ~~Q $\rightarrow$ NT~~
~~WA $\rightarrow$ SA~~ ~~SA $\rightarrow$ WA~~
 NT $\rightarrow$ WA WA $\rightarrow$ NT
 SA $\rightarrow$ Q Q $\rightarrow$ SA,
 WA $\rightarrow$ NT **WA $\rightarrow$ SA**
 ... ecc ...

Seleziono Q $\rightarrow$ NT: è consistente

Seleziono WA $\rightarrow$ SA: è consistente

Seleziono SA $\rightarrow$ WA

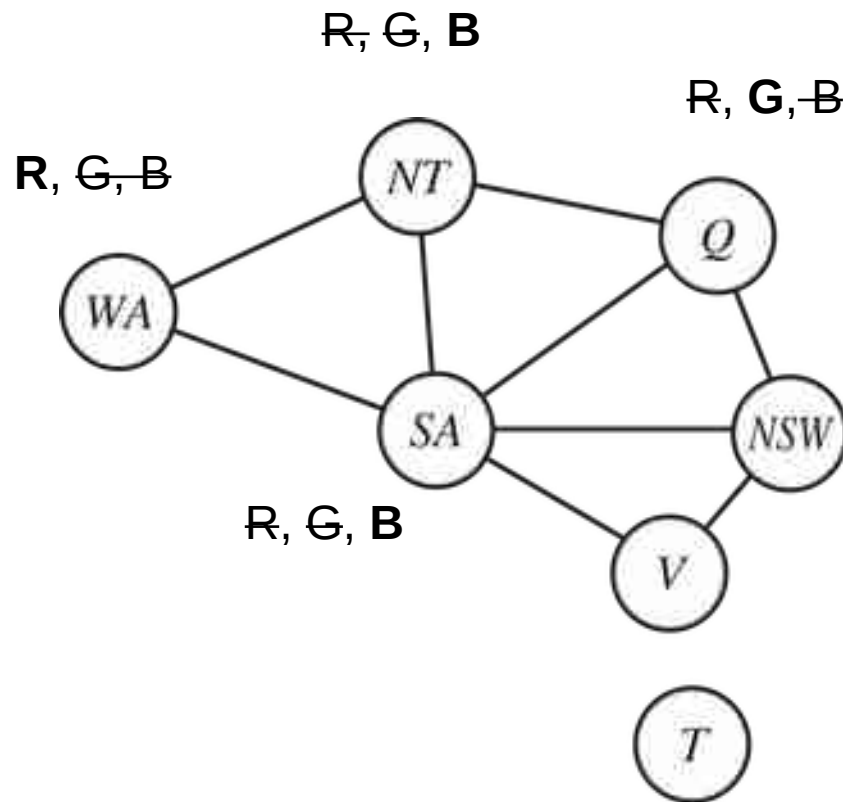
Non è arc-consistente:

il valore R di SA è inconsistente con
il valore R di WA

Togliamo R dai valori possibili di SA

Togliamo SA $\rightarrow$ WA da queue ma aggiungiamo
gli archi di tipo NODO $\rightarrow$ SA. Fra questi vi è
WA $\rightarrow$ SA che era stato rimosso prima

Esempio



QUEUE:

~~WA $\rightarrow$ NT~~ ~~NT $\rightarrow$ WA~~
~~NT $\rightarrow$ Q~~ ~~Q $\rightarrow$ NT~~
~~WA $\rightarrow$ SA~~ ~~SA $\rightarrow$ WA~~
~~NT $\rightarrow$ WA~~ ~~WA $\rightarrow$ NT~~
~~SA $\rightarrow$ Q~~ Q $\rightarrow$ SA,
WA $\rightarrow$ NT WA $\rightarrow$ SA
NT $\rightarrow$ SA
... ecc ...

Seleziono NT $\rightarrow$ WA: è consistente

Seleziono WA $\rightarrow$ NT: è consistente

Seleziono SA $\rightarrow$ Q

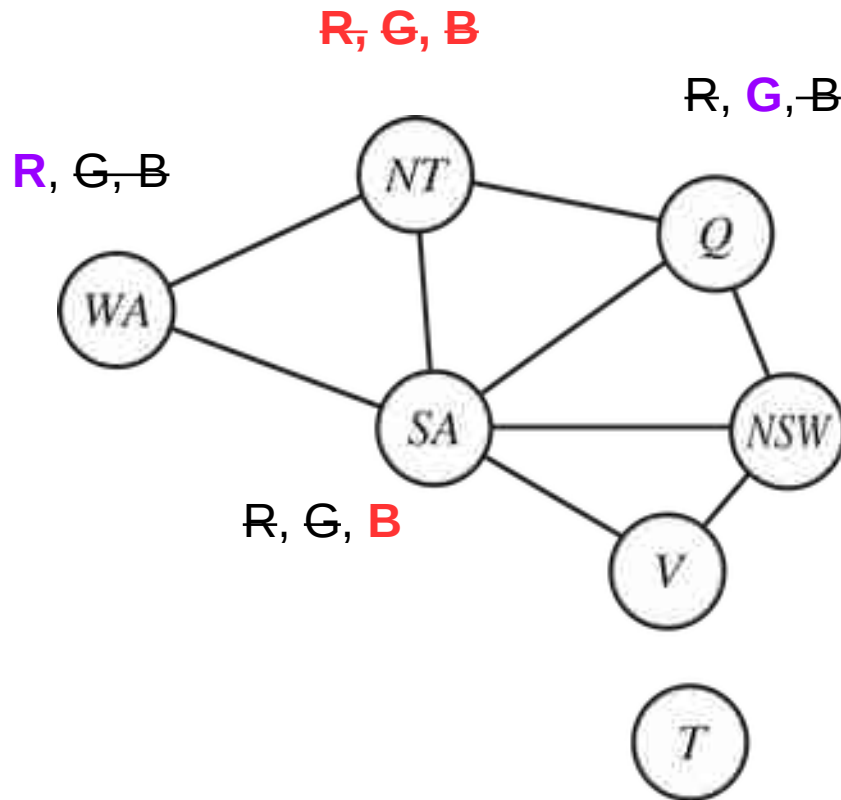
Non è arc-consistent:

il valore G di SA è inconsistente con
il valore G di Q

Togliamo G dai valori possibili di SA

Togliamo SA $\rightarrow$ Q da queue ma aggiungiamo
gli archi di tipo NODO $\rightarrow$ SA.

Esempio



QUEUE:

...
~~NT → WA~~ ~~WA → NT~~
~~SA → Q~~ ~~Q → SA~~,
~~WA → NT~~ ~~WA → SA~~
NT → SA
 ... ecc ...

Seleziono $Q \rightarrow SA$: è consistente
 Seleziono $WA \rightarrow NT$: è consistente
 Seleziono $WA \rightarrow NT$: è consistente
 Selezioni $NT \rightarrow SA$
 Non è arc-consistent:
 il valore B di NT è inconsistente con
 il valore B di SA

Togliamo B dai valori possibili di NT

NT ha dominio vuoto! L'assegnamento
 che stiamo verificando e cioè $WA=R, Q=G$
 è inconsistente

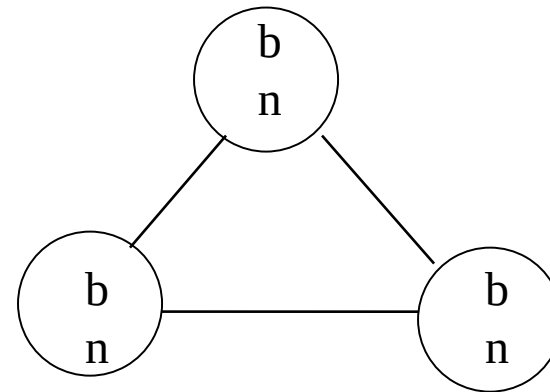
Commento su arc consistency

- La propagazione dei valori tramite arc consistency è un esempio di tecnica di inferenza
- La proprietà di arc consistency:
 - Consente di ridurre i domini delle variabili di molti CSP
 - Quando riduce i domini di tutte le variabili a un solo valore trova anche una soluzione
 - Quando rende vuoto un dominio scopre che un particolare CSP non può essere risolto
 - Di per sè non è però generalmente sufficiente a determinare una soluzione o ad accorgersi dell'irrisolvibilità di un CSP, vedremo un esempio per AC-3 (usata da sola è incompleta)

Incompletezza di AC-3

Esempio: colorazione con due colori

1	2
	3



È arc consistent eppure non c'è soluzione

Si può definire la **path consistency**, ma l'algoritmo aumenta di complessità

Valutazione di AC-3

- Un CSP con **n** variabili e **vincoli binari** ha al più **n^2** archi
- Sia **d** il numero massimo di valori di una variabile: Il tempo per la verifica della consistenza sarà nel caso peggiore **$O(n^2d^3)$**
- **Più costoso della verifica forward ma più efficace**
- **AC-3 è incompleto:**
vi sono alcuni assegnamenti inconsistenti che non vengono rilevati

Path consistency

- Proprietà più forte di arc consistency
- Identifica vincoli impliciti, inferiti da triplette di variabili
- Una coppia di variabili $\{X1, X2\}$ è **path consistent rispetto a una terza variabile** $X3$ quando:
 - 1) per ogni assegnamento $\{X1=a, X2=b\}$ consistente con i vincoli su $X1$ e $X2$,
 - 2) c'è un assegnamento di $X3$ che soddisfa i vincoli esistenti sulle coppie di variabili $\{X1, X3\}$ e $\{X2, X3\}$

Path consistency

A

B

$\{A=a, B=b\}$

In assenza di vincoli qualsiasi assegnamento di A e di B è consistente con una variabile C

C

A

B

$\{A=a, B=b\}$

Lo stesso assegnamento può invece avere un impatto quando A o B sono legate da vincoli a C

C

Path consistency, esempio

- Consideriamo il problema dell'Australia ma per semplicità supponiamo di avere solo **2 colori** (R e B)
- Consideriamo la coppia **{WA, SA}** e la variabile **NT**, Il CSP ha soluzione?
 - Ci sono due casi o {WA=R, SA=B} oppure {WA=B, SA=R}
 - Consideriamo i valori possibili per NT e rimuoviamo quelli che non rispettano la path consistency
 - Se {WA=R, SA=B}, NT non può essere R perché {WA=R, NT=R} non è consistente. Non può neanche essere B perché {SA=B, NT=B} non è consistente
 - Simili ragionamenti si possono fare anche partendo da {WA=B, SA=R}
 - Di conseguenza il CSP non ha soluzione
- L'algoritmo PC-2 (di Mackworth) propaga la proprietà di path consistency come AC-3 propaga quella di arc consistency

Generalizzazione: k-consistency

- Un CSP è k-consistent quando:
 - per ogni sottoinsieme costituito da $k-1$ delle sue variabili e ogni loro assegnamento consistente, è possibile individuare un assegnamento consistente per qualsiasi k-esima variabile
- 1-consistency: node consistency
- 2-consistency: arc consistency
- 3-consistency: path consistency

CSP fortemente k-consistent

- Un CSP è fortemente k-consistent quando è:
 - k-consistent,
 - (k-1)-consistent,
 - (k-2)-consistent,
 - ...,
 - 1-consistent

Risolvere CSP senza backtracking

- **È dimostrato che un CSP fortemente k-consistent può essere risolto senza dover mai fare backtracking**
- Intuitivamente:
 - Poiché è 1-consistent basta iniziare da un valore consistente per una variabile X_1
 - Poiché è 2-consistente è possibile individuare un valore consistente per le variabili direttamente in relazione con X_1
 - Poiché è 3-consistente è possibile individuare un valore consistente per le variabili in relazione con coppie delle precedenti
 - ...
- Complessità temporale: **$O(nd)$**
dove: **n**= numero di variabili, **d**=numero di valori del dominio
- Problema: la complessità temporale per decidere se un CSP è fortemente k-consistent è esponenziale

Vincoli speciali

- Alcuni tipi di vincoli sono usati di frequente e sono quindi stati studiati in modo specifico:
 - **Alldifferent**(X1, ..., Xn): i valori delle variabili elencate devono essere tutti differenti
 - **Atmost** (N, A1, ..., Ak): le attività A1, ..., Ak possono impegnare complessivamente al più N risorse

Alldifferent

- **Alldifferent($X_1, \dots, X_n$)**
- Se le n variabili possono assumere al più m valori distinti con $m < n$ il vincolo non può essere soddisfatto
- Altrimenti:
 - Si comincia ad assegnare un valore alle variabili che possono assumerne uno solo
 - Se il vincolo rimane soddisfatto si propagano le scelte riducendo via via i domini delle variabili rimanenti

Atmost: assegnamento di risorse

- **Atmost (N, A1, ..., Ak):**
 - 1) N è il numero di risorse disponibili
 - 2) A1, ..., Ak rappresentano le quantità di risorse assegnate alle varie attività
- Supponiamo che ciascuna variabile A_i abbia un dominio $\{v_{i1}, v_{i2}, \dots, v_{im}\}$, i cui valori sono ordinati in ordine crescente
- Si considerano i valori minimi $v_{11}, \dots, v_{k1}$ nei domini di A1, ..., Ak:
 - 1) Se la somma supera il limite imposto da atmost il vincolo non può essere soddisfatto
 - 2) Altrimenti si possono cancellare i valori massimi che non sono consistenti con quelli minimi degli altri domini

Atmost per domini su grandi numeri

- Applicazioni come la logistica richiedono di gestire grandi quantità di risorse: non è possibile scrivere esplicitamente array che enumerano tutti i possibili valori delle variabili
- In questo caso i domini sono rappresentati come intervalli $[v_{i1}, v_{im}]$
- La risoluzione del CSP passa attraverso la ridefinizione e la propagazione degli estremi di questi intervalli

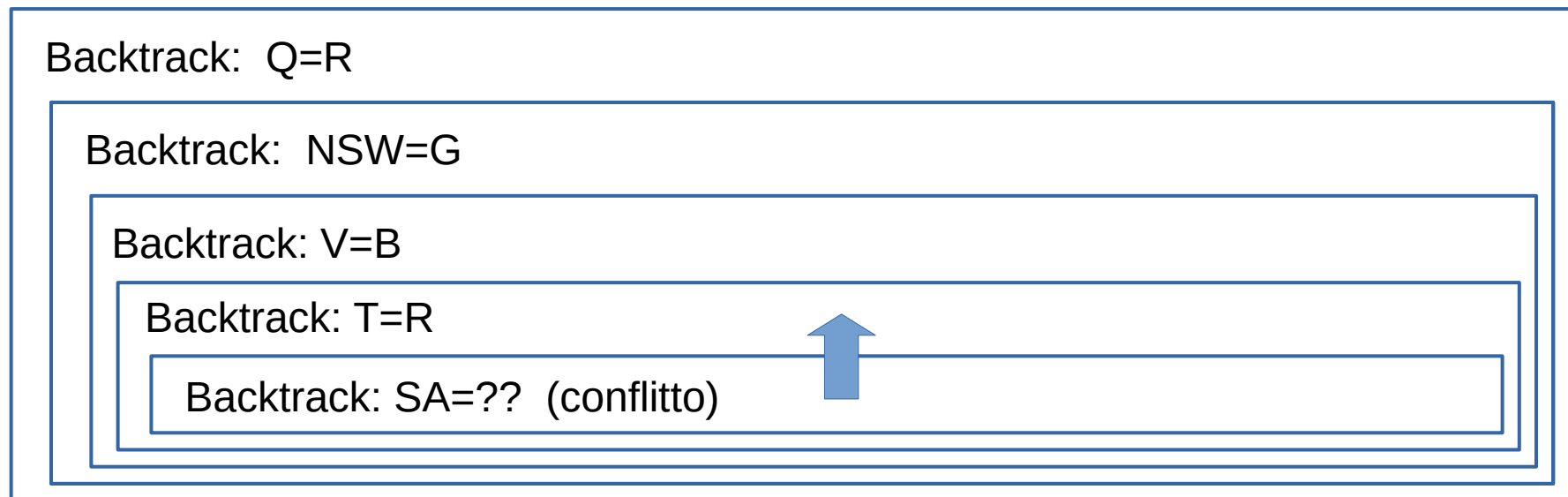
Migliorare ulteriormente il backtracking

Backjumping

Limite del backtracking

- Quando la ricerca con backtracking raggiunge un vicolo cieco, torna indietro alla variabile che era stata considerata al passo precedente (**backtracking cronologico**)
- Non sfrutta i vincoli, ha i limiti delle strategie di ricerca cieche
- Esempio:
 - Sia $\{Q=R, NSW=G, V=B, T=R\}$ l'assegnamento corrente
 - Scegliamo SA: tutti i valori vanno in conflitto (Q, NSW e V coprono lo spettro dei colori)
 - T è la variabile scelta alla chiamata ricorsiva precedente: si fa backtracking a questa variabile che però non crea il conflitto!!

Limite del backtracking: esempio



- Scegliamo SA: confina con Q, NSW e V. Dominio vuoto.
- T è la variabile scelta alla chiamata ricorsiva precedente: si fa backtracking a questa variabile che però non crea il conflitto!!

Backjumping: un backtracking non cronologico

- **Strategia più intelligente:**
fare backtracking a una variabile che potrebbe risolvere il problema (nell'esempio: V)
- Per ogni variabile occorre mantenere un insieme di assegnamenti che creano il conflitto (**conflict set**), nell'esempio {Q=R, NSW=G, V=B}
- **Backjumping:**
variante del backtracking che utilizza i conflict set per decidere a quale variabile ritornare in caso di vicolo cieco

Backjumping: conflict set

- Quando si raggiunge un vicolo cieco, il backjumping torna indietro alla variabile assegnata più di recente che ha un valore che partecipa a creare il conflitto, Gli assegnamenti che portano al conflitto costituiscono il **conflict set di SA**

- Definizione:*

Sia A un assegnamento parziale consistente, sia X una variabile non ancora assegnata. Se l'assegnamento $A \cup \{X=v_i\}$ risulta inconsistente per qualsiasi valore v_i appartenente al dominio di X si dice che A è un **conflict set** di X

Backjumping: conflict set minimo

- Un conflict set per una variabile è minimo quando togliendo uno qualsiasi degli assegnamenti che lo costituiscono non si ottiene più un conflict set
- Esempio: $\{Q=R, NSW=G, V=B\}$ è un conflict set minimo per SA
- Quando si ha un fallimento su una variabile il backjumping usa il suo insieme dei conflitti per identificare la variabile da restituire insieme all'indicazione di fallimento (nell'esempio: V)

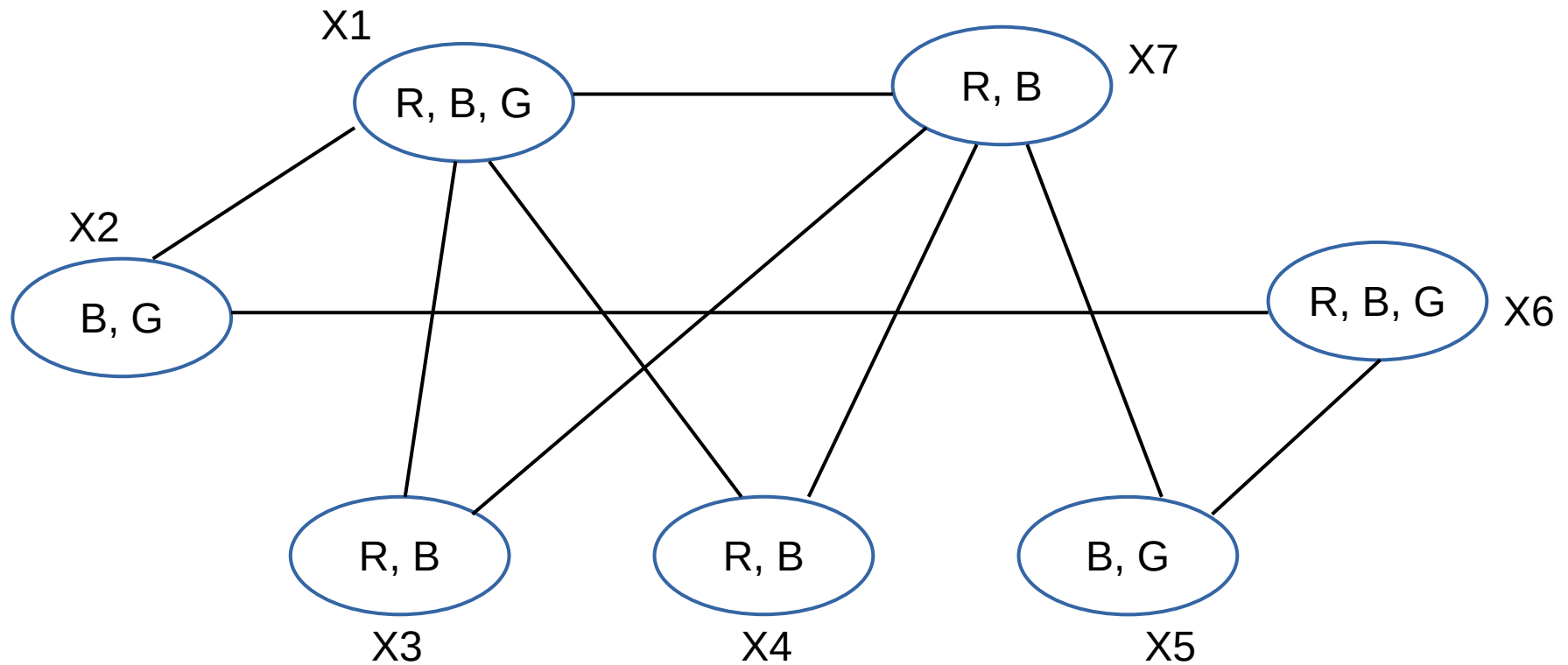
Backjumping (BJ) e forward checking (FC)

- Il FC può essere modificato in modo da costruire gli insiemi dei conflitti per ogni variabile:
 - Quando si assegna un valore a una variabile, FC propaga questa scelta attraverso i vincoli cancellando valori dai domini di altre variabili, esempio: $V=B$ cancella il valore B dal dominio di SA
 - basta arricchire FC in modo che registri la relazione fra la variabile assegnata e quelle che hanno subito una riduzione di dominio, quindi $\text{conflict}(SA) \leftarrow \text{conflict}(SA) \cup \{V=B\}$
 - **NOTA:**
FC rileva esattamente gli stessi conflitti rilevati da BJ, quindi l'uso di BJ congiunto a FC è ridondante

Non solo fallimenti: assegnamenti NOGOOD

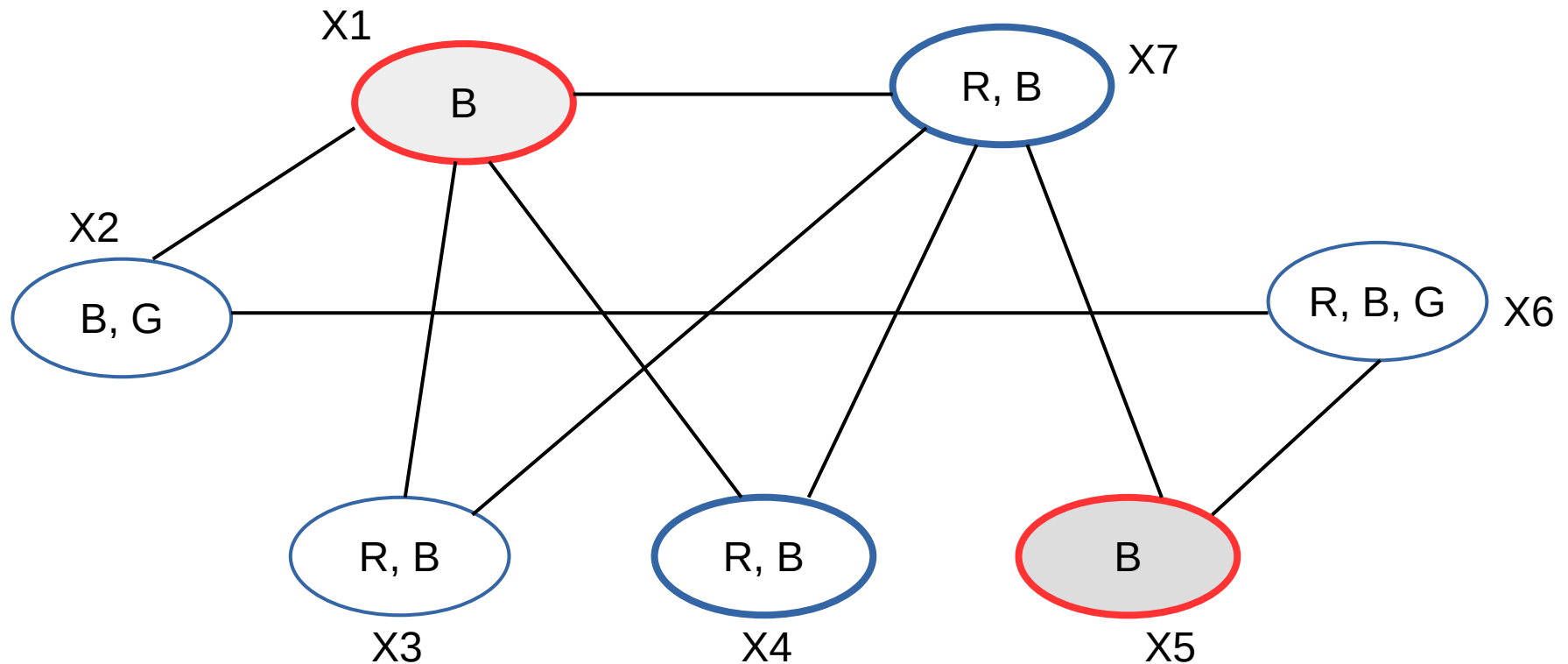
- Ricordate sistemi operativi e il deadlock?
- Ricordate le nozioni di stato sicuro, insicuro e di deadlock? Uno stato insicuro non corrisponde a un blocco ma porta necessariamente a un blocco
- l'analogo degli stati insicuri dei CSP è dato dagli **assegnamenti NOGOOD**: sono assegnamenti parziali che non appartengono all'insieme delle possibili soluzioni

Esempio



Vincoli: due variabili fra le quali c'è un vincolo non devono assumere lo stesso colore

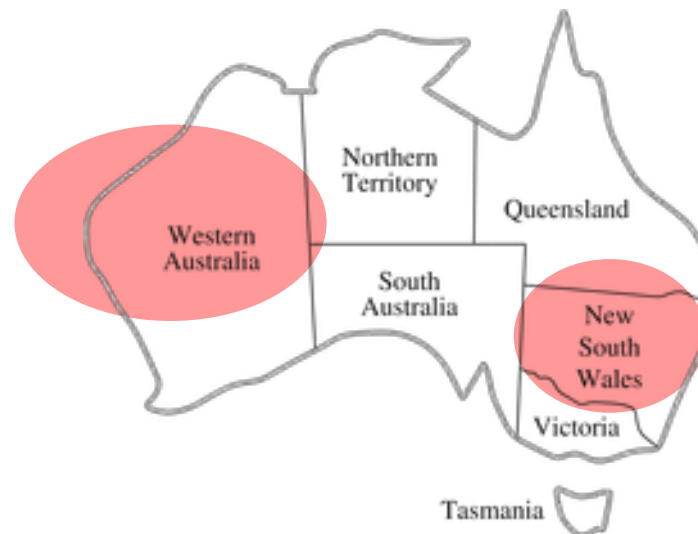
Esempio di stato NOGOOD



L'assegnamento parziale $\{X1=B, X5=B\}$ non è uno stato di fallimento ma è uno stato NOGOOD perché impedisce di trovare assegnamenti consistenti delle variabili $x4$ e $x7$.

NOGOOD per Australia

- **{WA=R, NSW=R}** è un assegnamento NOGOOD per il problema dell'Australia
- Non è un vicolo cieco, la ricerca può esplorare molte alternative, per esempio procederà con T=R
- Inoltre i conflict set di V, Q e NT conterranno o l'assegnamento di NSW o quello di WA, non entrambi
- Ma quando poi tenta di assegnare NT, SA, Q o V fallirà sempre (tutte dovranno essere o B o G): quando fallisce, come capire che la colpa era di questa scelta iniziale?



Conflict-directed backjumping

- Il problema non sussisterebbe se dovessimo assegnare valori solo a uno degli stati rimanenti, sussiste perché dobbiamo assegnarli a tutti e questi stati sono fra loro confinanti
- In altri termini il conflitto dipende da due fattori:
 - (1) dagli assegnamenti precedenti e
 - (2) dalle variabili rimaste a cui dovremo assegnare valori in futuro
- È possibile calcolare i conflict set in modo in modo tale che guidino in modo più efficace il backtracking (evitando di considerare $T=R$ da cui non dipende l'inconsistenza e saltando direttamente ai "colpevoli" veri)?

Australia

- Supponiamo di assegnare le variabili in quest'ordine:

WA = R;

NSW = R;

T = B;

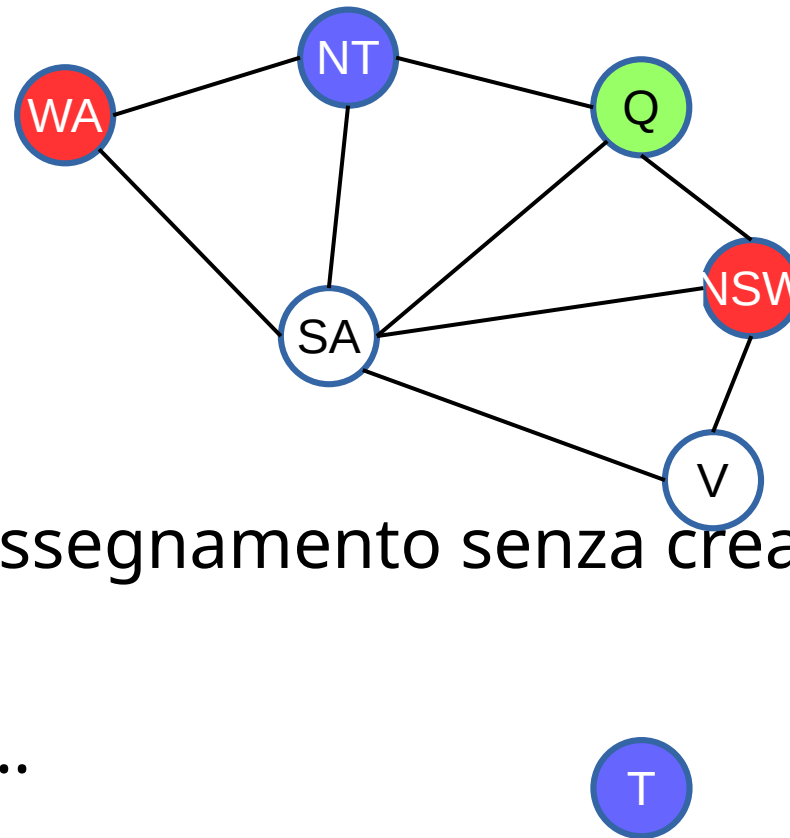
NT = B;

Q = G

- SA = ?**

Non è possibile fare l'assegnamento senza creare conflitti!

- Vediamo i conflict set ...



Australia. Conflict set

- Costruiti tramite FC come assegnamenti precedenti di variabili legate da un vincolo (si riportano solo i nomi di variabile per brevità). I seguenti conflict set sono stati costruiti man mano che si facevano gli assegnamenti:
 - $\text{conf}(\text{WA}) = \{\}$
 - $\text{conf}(\text{NSW}) = \{\}$
 - $\text{conf}(\text{T}) = \{\}$
 - $\text{conf}(\text{NT}) = \{\text{WA}\}$
 - $\text{conf}(\text{Q}) = \{\text{NSW}, \text{NT}\}$
 - **$\text{conf}(\text{SA}) = \{\text{WA}, \text{NSW}, \text{NT}, \text{Q}\}$**

Backjump e aggiornamenti dei conflict set

- **backjump:**

- Sia X_j la variabile corrente
- Indichiamo $\text{conf}(X)$ il conflict set della generica variabile X
- Se tutti i valori possibili di X_j falliscono si fa un backjump alla variabile X_i che è stata aggiunta a $\text{conf}(X_j)$ più di recente e si aggiorna $\text{conf}(X_i)$ in questo modo:

$$\text{conf}(X_i) \leftarrow \text{conf}(X_i) \cup \text{conf}(X_j) - \{ X_i \}$$

Australia

- Supponiamo di assegnare le variabili in quest'ordine:

WA = R;

NSW = R;

T = B;

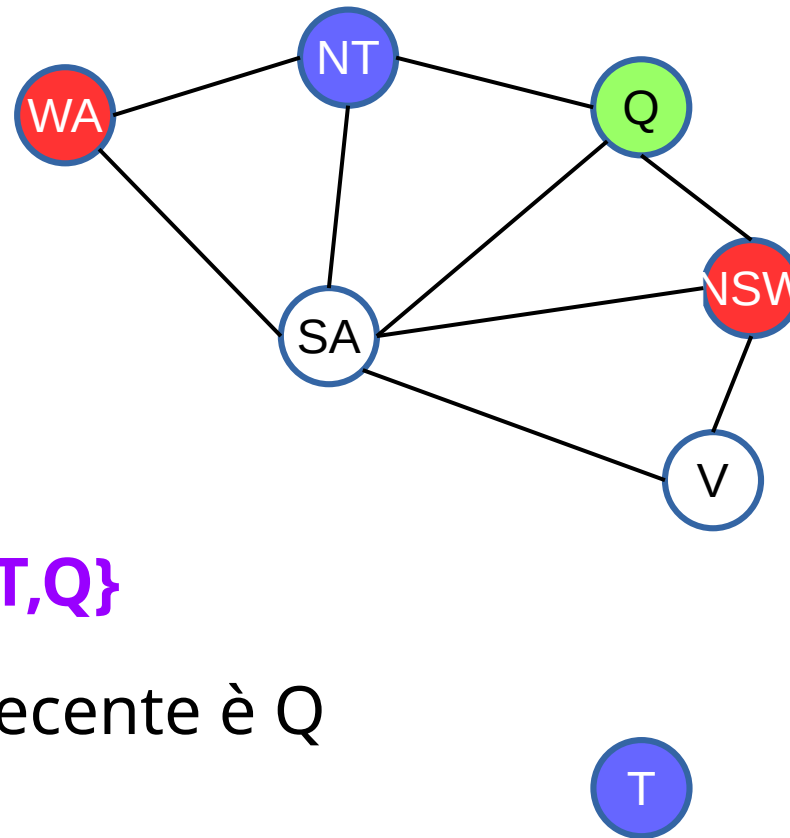
NT = B;

Q = G

- SA = ?**

conf(SA) = {WA,NSW,NT,Q}

- La vrb aggiunta più di recente è Q



Si parte da SA

- Backjump a Q:
 - $\text{conf}(Q) = \{\text{NSW}, \text{NT}\}$
 - Lo aggiorniamo arricchendolo dell'informazione di conflitto fornita da SA: $\text{conf}(Q) \leftarrow \text{conf}(Q) \cup \text{conf}(\text{SA}) - \{Q\}$
 $\text{conf}(Q) \leftarrow \{\text{NSW}, \text{NT}\} \cup \{\text{WA}, \text{NSW}, \text{NT}, Q\} - \{Q\}$
 - Quindi **conf(Q)** = {**WA**, NSW, NT}
- **NB:** WA non confina con Q ma prende parte al generarsi del conflitto e allora viene ricordato anche in associazione a Q. Viene rilevato un “vincolo implicito”
- Q=G si è dimostrato un assegnamento non consistente. Q=B e Q=R non sono possibili perché non si avrebbe arc consistency. Il dominio di Q diventa vuoto
- ...

Backjump a NT

- Backjump a NT:
 $\text{conf}(\text{NT}) \leftarrow \text{conf}(\text{NT}) \cup \text{conf}(\text{Q}) - \{\text{NT}\}$, quindi:
 $\text{conf}(\text{NT}) \leftarrow \{ \text{WA}, \text{NSW} \}$
- L'assegnamento $\text{NT}=\text{G}$ si è dimostrato non portare a soluzioni
- Si prova ad assegnare $\text{NT}=\text{B}$. Saltiamo i passi: l'assegnamento si dimostrerà altrettanto infruttuoso.
- Il dominio di NT diventa vuoto occorre quindi fare un backjump a NSW
- **$\text{conf}(\text{NSW}) \leftarrow \{ \} \cup \{ \text{WA} \}$ cioè: $\text{conf}(\text{NSW}) \leftarrow \{ \text{WA} \}$**
Il metodo ha scoperto la relazione fra NSW e WA!
- Tenta $\text{NSW}=\text{G}$ e arriverà a costruire una soluzione

- La simulazione è partita da uno stato NOGOOD
- Ha scoperto che si trattava di uno stato NOGOOD solo tramite l'esplorazione
- I vincoli hanno fornito una guida che ha permesso di evidenziare relazioni implicite fra le variabili
- Gli stati NOGOOD minimali possono essere registrati dall'algoritmo che potrà così evitare di ripetere questi assegnamenti in futuro
- Si tratta di una forma di **apprendimento**

Applicazioni

Esempio: Sudoku

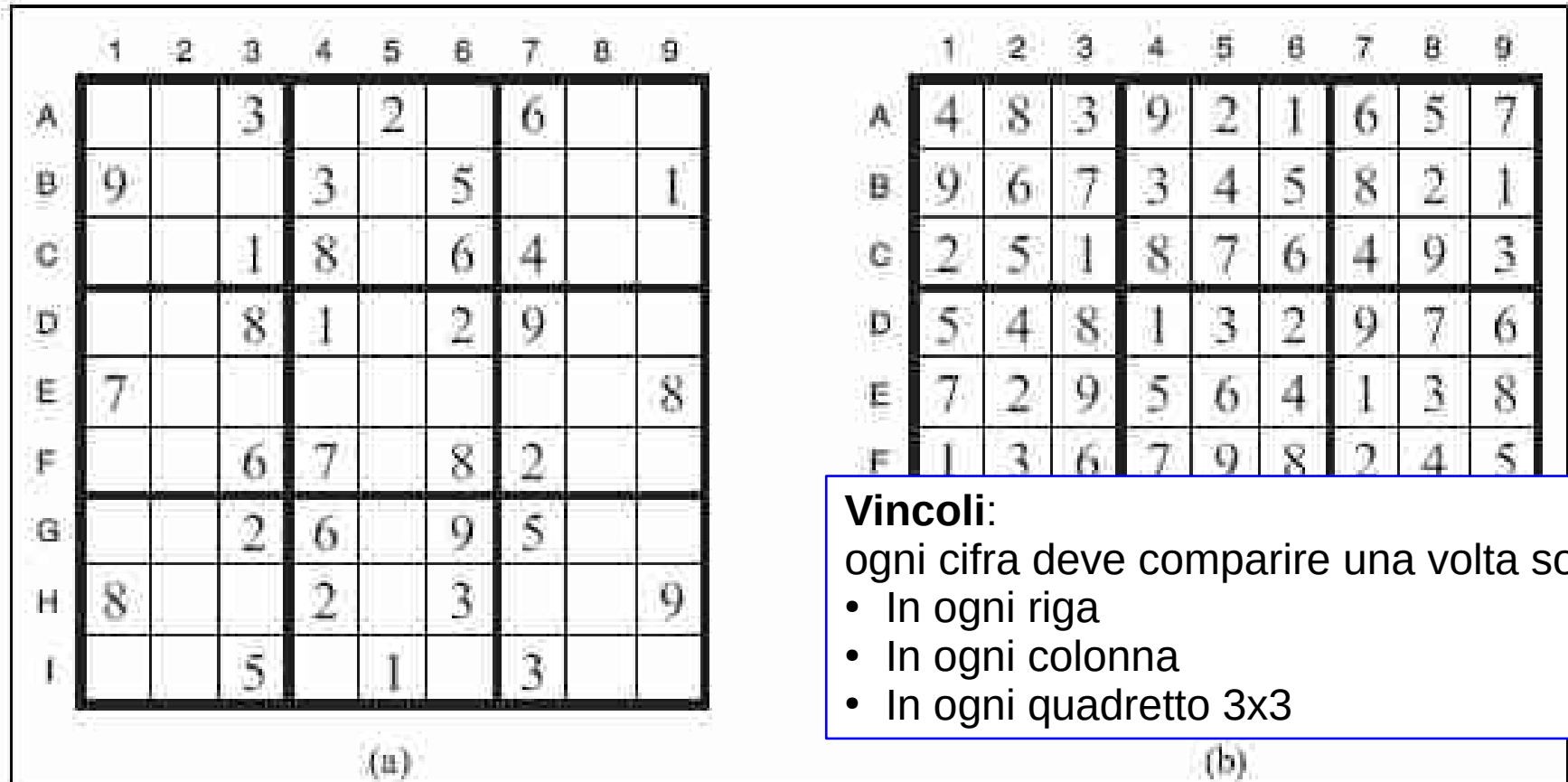


Figure 6.4
its solution.

FILES: figures/sudoku.eps (Tue Nov 3 13:49:46 2009). (a) A Sudoku puzzle and (b)

Quante regine riusciamo a gestire?

- Alcuni algoritmi attuali hanno risolto il problema delle N regine con $N=6.000.000$

Esempio: sudoku

- **Variabili:** ciascuna casella è rappresentata da una variabile x_{ij} , dove i è la riga e j la colonna che identificano la casella
- **Valori:** $\{1, \dots, 9\}$ alcune variabili hanno valore prefissato
- I valori prefissati sono **vincoli unari**
- **Altri vincoli:**
 - Sulle righe: per ogni i in $\{1, \dots, 9\}$ $\text{alldifferent}(x_{i1}, x_{i2}, \dots, x_{i9})$
 - Sulle colonne: per ogni j in $\{1, \dots, 9\}$ $\text{alldifferent}(x_{1j}, x_{2j}, \dots, x_{9j})$
 - Nei riquadri: sia S_{kl} il generico riquadro di lato 3, k e l compresi fra 0 e 2:
per ogni i, j dove $i = k*3+1$ e $j = l*3 + 1$
 $\text{alldifferent}(x_{ij}, x_{i(j+1)}, x_{i(j+2)}, x_{(i+1)j}, x_{(i+1)(j+1)}, x_{(i+1)(j+2)}, x_{(i+2)j}, x_{(i+2)(j+1)}, x_{(i+2)(j+2)})$

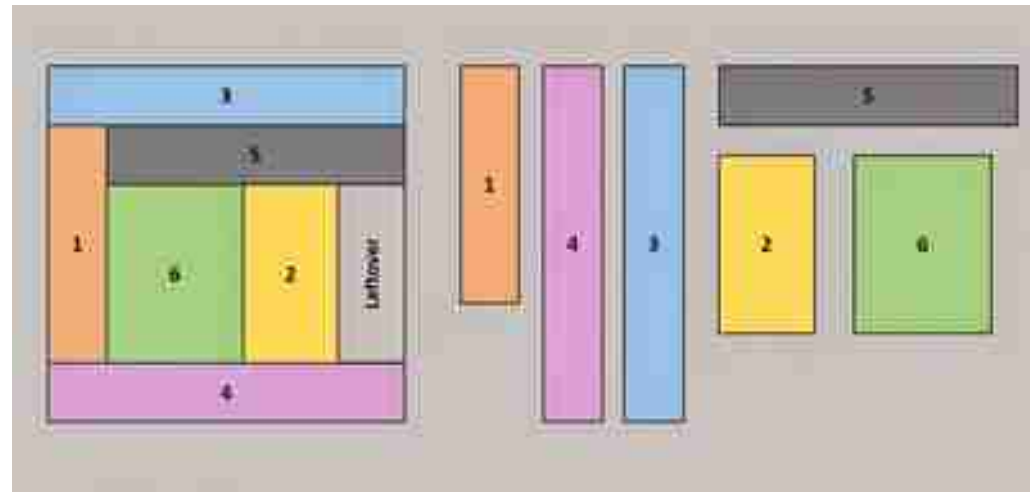
Applicazioni reali

I CSP si sono dimostrati vincenti in diversi contesti applicativi dello stesso genere di quelle affrontate in ricerca operativa, alcuni esempi:

- **Location of facilities** (es. warehouses): dato un elenco di magazzini e date delle richieste fatte da clienti, identificare un percorso che permetta di soddisfare le richieste visitando i magazzini così da minimizzare i costi
- **Job scheduling**: supponiamo che una fabbrica produca un range di prodotti, ogni tipologia richiede la sequenzializzazione di determinate operazioni svolte da macchinari. Ogni operazione richiede un tempo di esecuzione. Lo scopo è trovare una sequenza di produzione che minimizzi i tempi di produzione
- **Car sequencing**: nell'industria automobilistica la catena di montaggio assembla automobili partendo da un modello base a cui sono aggiunte caratterizzazioni scelte dai clienti (es. Optional). Automobili diverse avranno insiemi di optional diversi. Le auto sono portate da un nastro trasportatore e gli optional sono montati in aree di lavoro, ognuna delle quali ha una capacità massima. Il problema consiste nel definire un ordine di costruzione tale da non eccedere le capacità delle aree di lavoro

Applicazioni reali

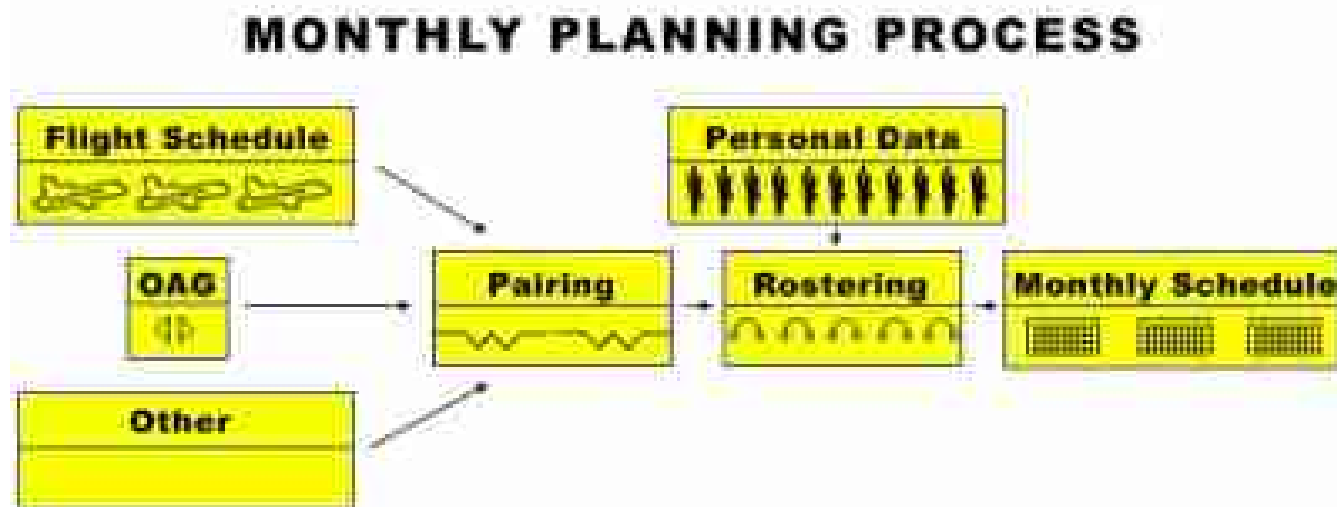
- **Cutting stock problem:** un materiale deve essere tagliato in pezzi più piccoli per un cliente minimizzando lo spreco



- **Vehicle routing:** n clienti vengono riforniti da uno stesso deposito. Il problema consiste nel trovare il percorso di costo minimo

Applicazioni reali

- **Timetabling:** costruzione automatica di orari (esempio per corsi di studi) tenendo conto dell'allocazione di risorse (aule, laboratori) e altri vincoli
- **Rostering e crew scheduling:** definizione di turni e di equipaggi (esempio: per compagnie aeree). Nel 1998 sono stati usati CSP realizzati in ECLiPSe per definire gli equipaggi dei treni delle ferrovie dello stato italiane



- I linguaggi di programmazione a vincoli sono dichiarativi e in parte sviluppati come moduli del linguaggio Prolog
- **ECLiPSe** (<http://eclipseclp.org/>):
un'introduzione passo passo alla programmazione in ECLIPSE della durata di circa 1:30 fatta da Sergii Dymchenko (in inglese):
<https://www.youtube.com/watch?v=84amHOgCEe8>
- **Febbraio 2015**: Opel vince il premio VDA (German Association of the Automotive Industry) Logistics Award grazie allo strumento di ottimizzazione della catena di distribuzione sviluppato in cooperazione con Flexis AG e basato su CSP ed ECLiPSe

- **CHR**: sviluppato dall'università di Leuven (Belgio)
<https://dtai.cs.kuleuven.be/CHR/download.shtml>
- Disponibile in diversi linguaggi di programmazione:
 - Vari prolog (SWI-prolog, CIAO, ecc.)
 - Java
 - Haskell
 - Linguaggio C
- Applicato per esempio alla navigazione robotica

- **SAT solvers**
- **Risolvono problemi a variabili booleane**
- Molti SAT solver sono basati sull'algoritmo DPLL arricchito con risoluzione di conflitti, backjumping, propagazione lineare, apprendimento di clausole e altre tecniche di ottimizzazione
- MiniSat (<http://minisat.se/>) vincitore della competizione 2005 è open source: "MiniSat is a minimalistic, open-source SAT solver, developed to help researchers and developers alike to get started on SAT"
- Sito della competizione annuale per SAT solver:
<http://www.satcompetition.org/>

- **GECODE**

<http://www.gecode.org/>

- Progetto open source con interfaccia grafica, basato su C++
- Molta documentazione che spiega tecniche, modelli, tutto nell'ottica di realizzare sistemi

- **GECODE/R**
- Gecode nel linguaggio Ruby
- Trovate implementati diversi esempi che abbiamo visto insieme ed altri:
<http://gecoder.rubyforge.org/examples/index.html>
- Trovate anche una miniguia alla modellazione di problemi come CSP:
<http://gecoder.rubyforge.org/documentation/modelling/index.html>

Answer set programming

- ASP non è un approccio specificamente pensato per risolvere CSP ma è uno strumento (concettuale e anche pratico) insegnato nel corso “Intelligenza Artificiale e Laboratorio” della laurea magistrale con cui si possono programmare e risolvere CSP
- Risolverete CSP come il problema della zebra